Source-Aware Trust Weighting and Cascade Detection: A Pipeline for Prompt Injection Detection in Large Language Model Applications

Khuu Hai Nguyen[1], Vo Ngoc Bao Tran[1], Nguyen Kim Ngan[1],

[1] Information Assurance Department, FPT University, Vietnam

*Correspondence: email@domain.edu.vn

Abstract

Background: Large Language Model (LLM)-based systems that ingest multiple untrusted input sources (user prompts, URLs, emails, files, RAG chunks) are exposed to a growing risk: prompt injection.

Methods: We present a five-stage detection pipeline — Input Classification, Source-aware Preprocessing, Source-aware Feature Extraction, Cascade Detection, and Risk Scoring — built around two core contributions: Source-aware Trust Weighting and Cascade Detection.

Results: *To be completed once results from notebooks 06–11 (baseline comparison, ablation study, adaptive attack, FPR, latency) are available.*

Conclusions: *To be completed.*

Keywords: Prompt Injection, Large Language Model, Cascade Detection, Trust-aware Feature Weighting, LLM Security

1. Introduction

Large Language Model (LLM)-based applications increasingly act on content the developer does not control: web pages fetched by a browsing agent, documents retrieved by a RAG pipeline, incoming emails, uploaded files, and the outputs of other tools. Because current LLMs process instructions and data through the same text channel, any of these sources can carry natural-language text that the model interprets as a command rather than as content to reason about — a vulnerability class known as *prompt injection* [1]. Two broad variants are distinguished in this paper, following the schema of Section 5.1: *direct* injection, where the attacker is the end user typing an adversarial instruction directly (e.g., "ignore all previous instructions"); and *indirect* injection, where the attacker embeds the instruction in third-party content that the application later retrieves and feeds to the model without the user ever seeing or authoring it [2]. Indirect injection is the more operationally dangerous variant in agentic deployments, since a single poisoned document, email, or search result can hijack the agent's behavior – exfiltrating data, taking unauthorized actions, or misinforming the user – without any malicious intent from the person actually using the system.

This paper presents a five-stage detection pipeline – Input Classification, Source-aware Preprocessing, Source-aware Feature Extraction, Cascade Detection, and Risk Scoring (Sections 4.1–4.5) – designed to handle both direct and indirect injection under a single architecture, and evaluates it end-to-end on two public benchmarks (Section 5.1) plus an out-of-distribution set and an adaptive/white-box attack suite (Sections 6.1–6.4). Rather than reporting only favorable results, every heuristic constant and architectural component in the pipeline is subjected to an explicit validation-set grid search or ablation, and negative or inconclusive findings are reported alongside positive ones (Section 9, Appendix A).

1.1. Contributions

- **A dual-stream detection architecture** that processes the `context` (external/retrieved) and `user_intent` (user-typed) channels independently at every stage from preprocessing through cascade inference (Sections 4.2–4.4), so that direct-injection samples with empty context (Section 5.1) are not left undetectable by a context-only design; this is the only ablated component to reach statistical significance on held-out data ($p = 0.008$, Section 6.2).
- **A three-stage Cascade Detection architecture** (rule-based pre-filter, DistilBERT, BERT with Mahalanobis out-of-distribution detection, Section 4.4) that reduces system-wide inference latency by 1.91× at p50 relative to running a single full BERT model on all traffic (Section 6.5), while achieving perfect recall on an out-of-distribution benchmark (Section 6.1).
- **An exhaustive, transparent hyperparameter-justification protocol** (Appendix A) applying validation-set grid search to every threshold and weight in the pipeline; several components originally proposed in the pipeline design – a keyword-density pre-filter, a source-aware Risk-Scoring feature bridge, and the BERT+OOD escalation stage's marginal accuracy contribution – are found, through this same protocol, to be inactive or statistically non-significant on the present data and are reported as such (Section 9) rather than omitted.

- **An adaptive, white-box robustness evaluation** (Section 6.4) showing that a technique with negligible static/black-box risk (homoglyph substitution, 28% Attack Success Rate) becomes the pipeline's largest attack surface once an attacker can iteratively query the deployed thresholds (84% ASR), and a cross-lingual probe showing recall collapses to 4% on machine-translated attack text – both failure modes invisible to the static, English-only benchmarks used in Section 6.1.

1.2. Scope

Only the Detection Pipeline (Steps 1–5, Sections 4.1–4.5) is evaluated experimentally in this paper. The Response Layer described in Section 7 (alerting, IP blocking, attacker profiling) is presented as a proposed system design and exercised only as an offline, in-memory simulation; neither BIPIA nor xTRam1 carries genuine request-level metadata (source IP, user agent), so its blocking and profiling logic could not be evaluated against real or realistic traffic (Section 7.3). All experimental results are restricted to English-language text (Section 9).

2. Related Work

2.1. Prompt Injection Attacks

Direct prompt injection – an end user directly instructing a model to ignore its system prompt or safety constraints – overlaps substantially with jailbreak research, and is the attack type represented by xTRam1 in this paper (Section 5.1). Indirect prompt injection, formalized and benchmarked by Yi et al. [2] (BIPIA) and by Liu et al. [1] (Open-Prompt-Injection), targets applications that retrieve and act on third-party content, and is the dominant attack type in this paper's primary dataset. Beyond plain-text injection, attackers can hide instructions from human reviewers while keeping them machine-readable – e.g., zero-width characters, homoglyphs, or PDF-layer tricks such as those in the *Hidden Semantic Injection* taxonomy of Liu and Ming [3], which motivates the deobfuscation and hidden-text detection mechanisms of Step 2 (Section 4.2).

2.2. LLM Guardrails and Detection Systems

Several detection systems target prompt injection directly. Meta's PromptGuard [4] is a lightweight, general-purpose classifier distributed in two sizes; DataSentinel [5] frames detection as a game-theoretic optimization using a fine-tuned Mistral-7B model. Both are evaluated as baselines in Section 6.1: on BIPIA/xTRam1-style indirect injection, PromptGuard v2 shows an extreme precision/recall imbalance (P= 0.9943, R= 0.0460) and DataSentinel's subsampled accuracy (0.5567) is close to chance despite high recall, consistent with both systems being tuned primarily for direct/jailbreak-style prompts rather than context-embedded indirect injection – the specific gap this paper's dual-stream design targets.

2.3. Benchmark Datasets

BIPIA [2] and xTRam1 [6] (used throughout this paper, Section 5.1) and Open-Prompt-Injection [1] (used for out-of-distribution evaluation, Section 6.1) are all binary-labeled, offline-evaluable datasets. AgentDojo is an agent-execution benchmark (tool-use trajectories with success/failure outcomes) rather than a binary-labeled classification dataset, and PINT is a proprietary benchmark (Lakera) without offline access; both were therefore out of scope for the baseline comparison in Section 6.1, as was PromptArmor, which requires GPT-4o API access unavailable to this study. These exclusions are logged as limitations (Section 9) rather than treated as equivalent coverage to the two benchmarks actually used.

3. Threat Model and System Architecture

3.1. Threat Model

The system under protection is an LLM application that assembles a single model context from multiple sources of heterogeneous trustworthiness: a developer-authored system prompt, the end user's own request, and zero or more pieces of external content (retrieved documents, emails, web pages, files) fetched automatically on the user's behalf. The attacker's goal is to have the model execute instructions the attacker authored rather than the ones the developer or user intended, by embedding those instructions in whichever of these channels the attacker can influence. Two attacker capability levels are considered: a *static, black-box* attacker who crafts an injection with a single transformation and no knowledge of the deployed detector (Section 6.4, static condition), and an *adaptive, white-box* attacker who knows the deployed cascade and OOD thresholds and can iteratively re-query the pipeline to find an evasive variant (Section 6.4, adaptive condition). The pipeline does not assume a trusted execution boundary beyond the detection system itself: no component in this paper's evaluation scope (Section 1) blocks or sandboxes the model's downstream tool calls, since that responsibility is assigned to the proposed, unevaluated Response Layer (Section 7).

3.2. Trust Levels and Attack-Type Weighting

Every input component is assigned a categorical *trust level* at classification time (Table 1, Section 4.1), reflecting how much control the developer has over that channel, and an *attack-type weight* W_{attack} distinguish-

ing Direct, Indirect, and Data-Exfiltration subtypes (Section 4.1). These two signals feed the Risk Scoring formula (Eqs. 2–4, Section 4.5) via the trust multiplier M_{trust} and attack-type weight W_{attack} respectively: a low-trust channel carrying a high-confidence attack prediction is scored more severely than the same confidence on a high-trust channel, reflecting the asymmetric cost of misclassifying content the developer never intended to be instructions.

3.3. System Architecture

Figure 1 shows the six-stage architecture. Stages 1–5 form the *Detection Pipeline*, evaluated experimentally in Sections 4.1–4.5: Input Classification assigns source type and trust level; Source-aware Preprocessing deobfuscates and generates the dual analysis/sanitization streams; Source-aware Feature Extraction produces a weighted lexical/semantic/source feature vector; Cascade Detection produces a label, confidence, and OOD flag via a three-stage rule/DistilBERT/BERT+OOD escalation; and Risk Scoring converts these signals into a bounded score and a four-level decision (Allow/Monitor/Warn/Block, Table 20). Stage 6, the Response Layer (Section 7), is a proposed design that would consume the Stage-5 decision to drive alerting, IP blocking, and attacker profiling in a deployed system; it is exercised only as an offline simulation in this paper (Section 1, Scope).

4. Methodology — Detection Pipeline

4.1. Step 1: Input Classification

Step 1 determines the source type of each input component (`context` and `user_intent`), assigns the corresponding *trust level*, and computes preliminary anomaly signals for the audit trail. The entire step is **rule-based** (regex/pattern matching), with no machine-learning model invoked, so it can run offline over all 77,314 samples with no network side effects.

4.1.1 Rule-based Input-Type Classifier

The `classify_input_type()` function applies a prioritized rule chain (Algorithm 1): (1) System Prompt marker, (2) URL pattern, (3) Email header pattern, (4) file extension or code fence, (5) special RAG tag, (6) an email address occurring in free text, (7) a fallback to RAG Chunk driven by the `is_retrieved` flag, and (8) User Prompt as the final fallback. This ordering is deliberate: URL is prioritized over email because, in RAG/web content, a URL is a stronger signal; a bare email address in text is only considered after ruling out stronger structural cues (real email headers, file markers, RAG tags), reducing false matches.

The resulting input type is mapped to a *trust level* according to Table 1, consistent with the design presented in Section 3.

Algorithm 1 `classify_input_type`

Require: text t, flag `is_retrieved`

if t matches SystemPromptMark **then return** SystemPrompt
end if
if t matches UrlPattern **then return** Url
end if
if t matches EmailHeaderPattern **then return** Email
end if
if t matches FileExtPattern or CodeFencePattern **then return** File
end if
if t matches RagTagPattern **then return** RagChunk
end if
if t matches EmailAddrPattern **then return** Email
end if
if `is_retrieved` = True **then return** RagChunk
end if
return UserPrompt ▷ final fallback

4.1.2 Direction and Attack Subtype Tagging

In parallel with input-type classification, `detect_direction()` locates *attack cues* (e.g., *ignore all previous instructions*, *you are now*, *reveal your prompt*) via the AttackCuePattern regular expression, **without** using the ground-truth label, to avoid circular reasoning. If a cue appears in `user_intent` ⇒ `direct`; if in `context` ⇒ `indirect`; if both ⇒ `both`. A separate DataExfiltrationPattern isolates a `data_exfiltration` subtype (requests to send/embed data externally) from generic `indirect` cases — a pattern characteristic of BIPIA samples that embed malicious URLs in retrieved content.

Tables 2 and 3 cross-tabulate the rule-inferred `direction`/`attack_subtype` against the ground-truth `attack_type` (`indirect` = BIPIA, `direct` = xTRam1), measured over all 77,314 samples.

Only 3.4% of injection samples and 0.6% of benign samples have `direction` $\neq$ `none` — indicating that the rule-based cues in Step 1 achieve good *precision* (a matched cue is a reliable injection signal) but very low *recall* when used alone, motivating the need for a machine-learning model at Step 4 (Cascade Detection, Section 4.4) rather than relying on rules alone. Similarly, `has_nested = True` occurs in 15.0% of injection samples versus 2.8% of benign samples.

4.1.3 Nested Content Detection (Single Level)

`detect_nested_content()` scans each `context` once for an embedded input type of a different kind (e.g.,

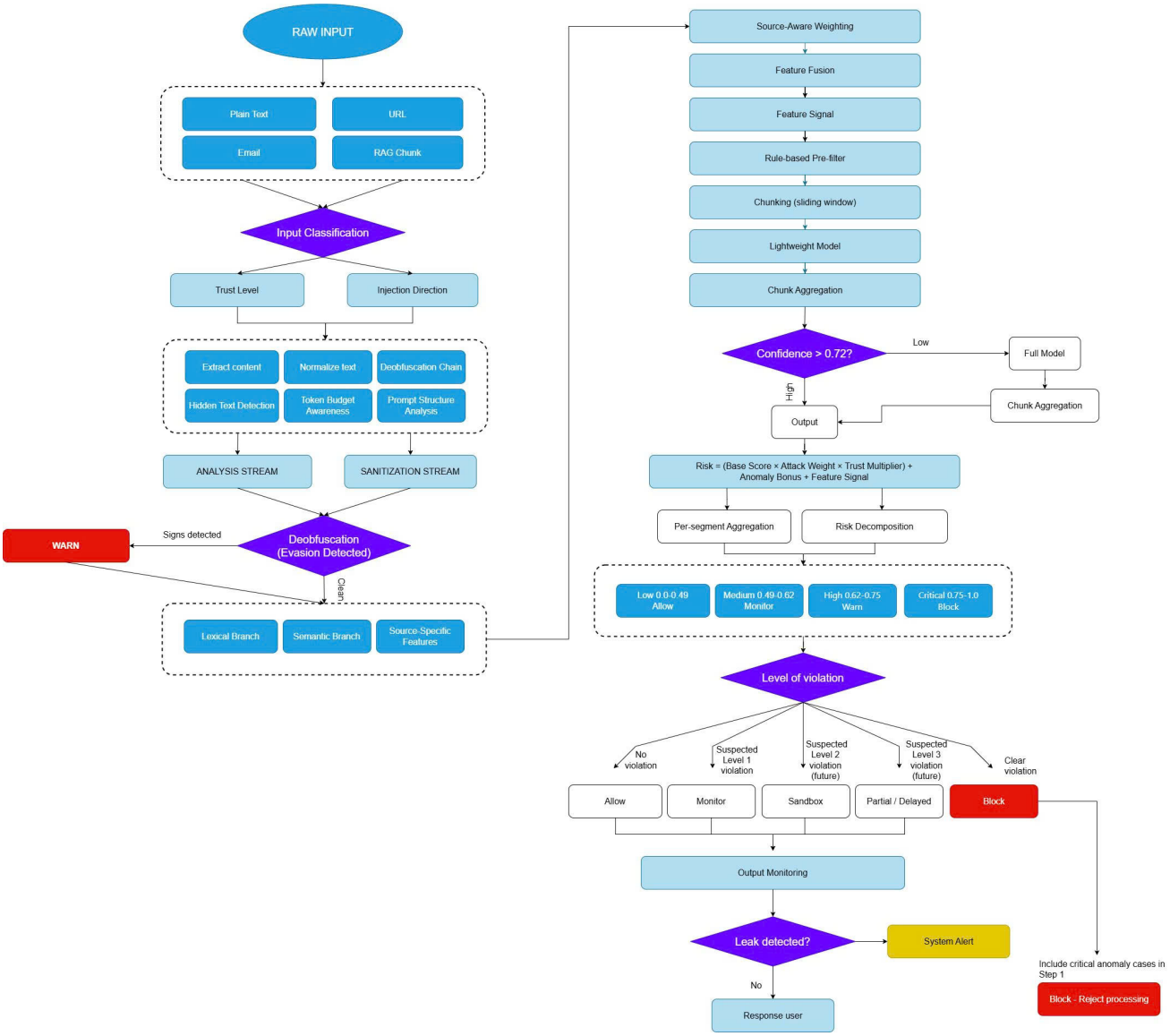


Figure 1: Six-stage pipeline architecture. Stages 1–5 (Input Classification, Source-aware Preprocessing, Feature Extraction, Cascade Detection, Risk Scoring) form the Detection Pipeline evaluated in this paper; Stage 6 (Response Layer) is a proposed design evaluated only as an offline simulation (Section 7).

an email containing a code block, or a file containing a URL), **without recursion** — limited to one level, consistent with the scope declared in Limitations (Section 9); handling multi-level nesting is left to Future Work (Section 10).

4.1.4 Audit Trail

Each sample is attached to an `AuditTrail` record with seven required fields: `timestamp`, `session_id`, `source_ip`, `user_agent`, `input_type`, `trust_level`, and `anomaly_flags` — serving both the detection layer and the (optional) proposed Response Layer in Section 7.

4.1.5 Hyperparameter Validation: ATTACK_CUE_PATTERNS

As required for justifying all heuristic constants (Appendix A), the 20 phrases in AttackCuePattern were evaluated independently on the validation set (7,730 samples, *test set untouched*), treating each pattern as a standalone binary classifier (Table 4, abbreviated — the full 20-pattern table is provided in the Appendix).

Fifteen of the 20 patterns achieved precision ≥ 0.5 but very low recall (each pattern captures only a small fraction of injection samples), consistent with their role as *supporting pre-filter/anomaly signals* rather than a primary detector. The four patterns with precision < 0.3 (*system prompt*, *your new task*, *your role is*, *assume the role*) were removed to reduce false positives in the `role_switch_keyword` signal used at Step 5; the remaining 11 patterns with support < 10 on the val-

Table 1: Input Type → Trust Level mapping.

Input Type	Trust Level	Rationale
System Prompt	High	Controlled by the developer
User Prompt	Medium	The user may intentionally attack
Email / URL / File	Low	Content originates externally
RAG Chunk	Very Low	Auto-loaded by the system, invisible to the user

Table 2: Ground-truth `attack_type` × rule-inferred `direction`.

attack_type	direct	indirect	none
direct (xTRam1)	757	0	9,380
indirect (BIPIA)	0	755	66,422

Table 3: `direction` vs. ground-truth label (0: benign, 1: injection).

direction	label = 0	label = 1
direct	135	622
indirect	95	660
none	39,013	36,789

idation set are flagged as inconclusive and require re-evaluation once more data is available (see Limitations, Section 9).

4.1.6 Gate 1 Verification

All eight Gate 1 criteria (correct trust-level mapping for both `context` and `user_intent`, a complete 7-field audit trail, the `split` column preserved, xTRam1's empty context correctly mapped to `user_prompt`/`medium`, sufficient Direct and Indirect data, and the Data Exfiltration subtype correctly separated) **passed** on all 77,314 samples, confirming the data is ready to proceed to Step 2 (Preprocessing, Section 4.2).

4.2. Step 2: Source-aware Preprocessing

Step 2 takes the output of Step 1 (`step1_input_classification.csv`, 77,314 rows) and applies five processing groups to both `context` and `user_intent`: (1) a deobfuscation chain, (2) hidden-text detection, (3) prompt-structure analysis, (4) context-flooding ratio computation with sliding-window chunking, and (5) dual-stream generation. All processing is deterministic and rule-based; no model inference is required at this stage.

4.2.1 Deobfuscation Chain

`deobfuscate()` applies, in order: zero-width character removal, homoglyph normalization (Cyrillic/Greek look-alikes → Latin), HTML/URL entity decoding, selective leetspeak decoding (only tokens whose decoded form matches a known attack keyword, to avoid blind substitution on ordinary text), and base64 candidate detection (kept only if the decoded payload is printable and semantically plausible, filtering the many false positives produced by naive regex matching on English text). Table 5 reports the prevalence of each real signal across the full dataset (77,314 rows).

The dataset contains no genuine base64 payloads or hidden CSS styles, confirming that BIPIA/xTRam1 do not include these obfuscation classes; the functions remain generically applicable and were validated on synthetic and real corpora (below).

4.2.2 Hidden-Text Detection

`detect_html_hidden_style()` flags markup-level hiding (`display:none`, `visibility:hidden`, `font-size:0`, `opacity:0`). `detect_pdf_hidden_text()` is a generic text-layer-vs-OCR-layer similarity comparator, thresholded at 0.6 by default (Section A).

Because BIPIA/xTRam1 contain no binary PDF files, a dedicated **real, binary PDF corpus** was generated from the validation split (not the test set) to validate this threshold under genuine conditions, rather than relying purely on synthetic strings. Following the *Hidden Semantic Injection* taxonomy (EG04–EG14) of Liu and Ming [3], five embedding techniques were implemented: `white_on_white`, `tiny_font`, `off_page`, `zero_height_matrix`, and `hidden_ocg_layer` (a PDF Optional-Content-Group set to OFF by default). For each technique, 60 PDF pairs (label 0/1) were rendered from validation-split sentences (300 files total: 150 hidden, 150 benign); text layers were extracted with `pdfplumber` and OCR layers with EasyOCR over PyMuPDF-rendered pages.

The best F1 (0.7075) occurs at threshold 0.95, well above the documented default of 0.6, which only

Table 4: Validation results for representative patterns in ATTACK_CUE_PATTERNS on the validation set ($n = 7{,}730$).

Pattern	Precision	Recall	F1	Support
"you are"	0.6031	0.0208	0.0401	131
"disregard"	0.9643	0.0071	0.0141	28
"you must"	0.8125	0.0068	0.0135	32
"you will"	0.4407	0.0068	0.0135	59
"act as"	0.5185	0.0037	0.0073	27
system prompt†	0.0000	0.0000	0.0000	0
your new task†	0.0000	0.0000	0.0000	0
your role is†	0.0000	0.0000	0.0000	0
assume the role†	0.0000	0.0000	0.0000	0

† Four patterns with precision < 0.3, removed from AttackCuePattern in `classifiers.py` after this validation round.

Table 5: Prevalence of deobfuscation/structural signals on the full dataset ($n = 77{,}314$).

Signal	Rows
Zero-width characters	114
Homoglyphs	1
Base64 (meaningfully decoded)	0
Any HTML tag	6,358
Real hidden CSS style	0
Delimiter injection	14,373
Role-switch token	10

Table 6: Grid search for the hidden-text similarity threshold on the real PDF corpus ($n = 300$, validation-split only).

Threshold	Precision	Recall	F1
0.95	0.5474	1.0000	**0.7075**
0.90	0.5612	0.8867	0.6873
0.85	0.5635	0.7400	0.6398
0.80	0.5610	0.6133	0.5860
0.75	0.5548	0.5400	0.5473
0.70	0.5484	0.4533	0.4964
0.65	0.5800	0.3867	0.4640
0.60 (default)	0.5500	0.2933	0.3826

reaches F1 = 0.3826 on the real corpus. Mean similarity for hidden samples ranges from 0.646 (`tiny_font`) to 0.729 (`white_on_white`), all clustering near 0.6–0.73 – explaining why a 0.6 cutoff under-detects. Given this evidence, we retain 0.6 only as a conservative interim default and flag it for revision to a higher value (0.85–0.95) pending a larger real corpus (see Section A and Limitations).

4.2.3 Prompt-Structure Analysis

`analyze_prompt_structure()` extends the coarse delimiter count from Step 1 by separately tracking seven delimiter families (triple-quote, hash/dash/equal fences, code fence, chat-template tokens, XML-like role tags) plus role-switch tokens (`system:`, `user:`, ...), producing a heuristic `structure_risk_score` used as an input feature at Step 3.

4.2.4 Context-Flooding Ratio and Chunking

`compute_context_flooding_ratio()` estimates `token_count`/`context_window` (default window 512) using the real DistilBERT tokenizer when available, falling back to a word-count $\times 1.3$ estimate otherwise. Because BIPIA contexts can reach 26,406 characters, feeding them directly to DistilBERT/BERT would silently truncate and could drop an injection payload located mid- or end-of-text. `chunk_for_model()` (Algorithm 2) addresses this by splitting the *unmodified* `analysis_stream` into overlapping, model-sized chunks (stride = 50 tokens) so that no payload straddling a chunk boundary is fully lost.

Applied to the full dataset, 9,844 rows (12.73%) exceed a single 512-token window and are chunked; the longest context (26,406 characters, the case referenced above) is split into 36 overlapping chunks. Crucially, `context_pp_analysis_stream` is verified to retain 100% of the original character length on every row, independent of chunking, since it feeds the length-unconstrained rule pre-filter at Step 4; only the chunked representation (`context_pp_chunks_json`) is passed to DistilBERT/BERT, which must aggregate

Algorithm 2 `chunk_for_model` (sliding-window chunking)

Require: text t, context window w (default 512), stride s (default 50), reserved special tokens r (default 2)

usable $\leftarrow w - r$; step $\leftarrow$ usable $- s$
Tokenize t (real tokenizer if available, else word-based fallback with offset mapping)
$i \leftarrow 0$; chunks $\leftarrow [\,]$
while $i < n$ **do** ▷ n = total token count of t
  $j \leftarrow \min(i + \text{usable}, n)$
  Append chunk spanning tokens $[i, j)$, with character offsets, to chunks
  if $j \geq n$ **then break**
  end if
  $i \leftarrow i + \text{step}$
end while
return chunks, |chunks|, (|chunks| > 1)

predictions across chunks (e.g., max-pooling) rather than using only the first chunk.

4.2.5 Dual-Stream Generation and Verification

`make_dual_stream()` produces two parallel representations: an *analysis stream* (the untouched original text, used by Steps 3–4 for detection) and a *sanitization stream* (base64 payloads redacted, delimiter sequences escaped, role-switch tokens quoted as literal data), the latter reserved for the LLM only if the final decision is *Allow*. Table 7 reports the four-part Gate 2.1 verification performed on the full dataset.

4.2.6 Context-Flooding Threshold Validation

As with hidden-text detection, no genuine context-flooding attacks exist in BIPIA/xTRam1. A synthetic set of 300 samples (150/150) was generated by padding validation-split sentences with filler tokens to 1.5–3× the context window. Table 8 shows the resulting grid search.

The synthetic-optimal threshold (1.9, F1 = 0.9934) is substantially higher than the documented default of 0.7 (F1 = 0.8955), indicating that 0.7 is comparatively aggressive and may over-flag long-but-legitimate context. Because the synthetic construction (filler-token padding) inflates separability, this result is reported strictly as a directional signal, not a final calibration; we retain 0.7 as a conservative interim default pending real flooding-attack data (Section 9).

4.2.7 Gate 2 Verification

All ten Gate 2 criteria passed: correct dual-stream separation, error-free deobfuscation on the full dataset, functioning HTML/PDF hidden-text detection, complete flooding-ratio computation, both grid searches confined to the validation split, the test set untouched throughout the notebook, the `split` column preserved, and zero payload loss under chunking – confirming readiness for Step 3 (Feature Extraction, Section 4.3).

4.3. Step 3: Source-aware Feature Extraction

Step 3 consumes the output of Step 2 (`step2_preprocessing.csv`, 77,314 rows, 68 columns) and produces three feature groups — lexical, semantic, and source-specific — fused with source-aware weighting, yielding the input matrix for Step 4.

4.3.1 Lexical Features

`extract_lexical_features()` computes 24 scalar features per text: token/unique-token counts, average/min/max token length, lexical diversity, Shannon entropy, uppercase/digit/punctuation ratios, keyword count/density/first-position (against a 49-term AttackKeywords set), imperative-verb ratio (21-term ImperativeWords set), delimiter count, URL/email count, word-frequency statistics, a heuristic `suspicious_pattern_score` (Eq. 1), and four TF-IDF summary statistics.

$$\text{score} = 0.45\, d_{kw} + 0.35\, r_{imp} + 0.20\, \min\left(\tfrac{n_{delim}}{10}, 1\right) \quad (1)$$

where d_{kw} is keyword density and r_{imp} is imperative ratio. The weights (0.45/0.35/0.20) are heuristic and, consistent with Section A, have not yet been grid-searched.

A dedicated TF-IDF vectorizer ($\leq$ 5,000 features, 1–2-gram) is fit *only on the training split* (61,849 rows → 115,590 non-empty context/user_intent texts combined) to avoid leakage; the resulting 5,000-token vocabulary is reduced to four summary statistics per text (sum, max, mean-nonzero, nonzero-ratio) rather than kept as raw sparse columns, to control dimensionality relative to the ∼60 handcrafted features. A custom stop-word list is used instead of scikit-learn's default English list: negation terms (*not*, *never*, *no*, *cannot*, ...) and all attack/imperative keywords are explicitly retained, since removing negation would erase signals such as *"do **not** follow the instructions"*. This was verified post-fit: both negation terms and the keyword *ignore* remain in the fitted vocabulary.

4.3.2 Semantic Features

Embeddings are generated with `SentenceTransformer(all-MiniLM-L6-v2)` (384 dimensions). For each text, five embedding statistics (mean, std, max, min, L_2 norm) are computed, along with three attack-similarity scores against a fixed set of ten hand-written attack prototype sentences (e.g., *"Ignore previous instructions."*, *"Reveal the*

Table 7: Gate 2.1 verification of the dual-stream mechanism (full dataset, $n = 77{,}314$).

Check	Result
`analysis_stream` identical to original text	PASS (100%)
Flagged rows ($n = 14{,}486$) where `sanitization_stream` $\neq$ `analysis_stream`	14,486 (100.0%)
Unflagged rows ($n = 62{,}828$) where `sanitization_stream` = `clean_text`	62,828 (100.0%)
Chunked rows ($n = 9{,}844$) with zero payload loss (full character coverage across chunks)	9,844 / 9,844 (100.0%)

Table 8: Grid search for the context-flooding ratio threshold on synthetic validation data ($n = 300$).

Threshold	Precision	Recall	F1
1.9	0.9868	1.0000	**0.9934**
1.7	0.9804	1.0000	0.9901
1.5	0.9740	1.0000	0.9868
1.3	0.9554	1.0000	0.9772
1.0	0.8876	1.0000	0.9404
0.70 (default)	0.8108	1.0000	0.8955
0.50	0.7177	1.0000	0.8357
0.30	0.6048	1.0000	0.7538

hidden system prompt."): maximum cosine similarity, average similarity, and mean of the top-3 similarities. A `context_similarity` score (cosine similarity between the `context` and `user_intent` embeddings) and a derived `semantic_anomaly_score` (1 − context_similarity) are also computed, on the hypothesis that a context semantically unrelated to the user's request is a signal of indirect injection.

4.3.3 Source-Specific Features and Source-Aware Weighting

`extract_source_specific_features()` computes source-type-specific counts (email/attachment keywords, URL/HTML-tag/hidden-style counts, code-block/markdown/base64 counts, role-switch and delimiter counts) and reuses Step 2's outputs directly (`context_pp_has_hidden_style`, `context_pp_flooding_ratio` > 0.7, `context_pp_role_switch_hits`, `context_pp_total_delimiter_hits`) rather than recomputing them, to keep Steps 2–3 consistent. Table 9 reproduces the source-aware weighting table used in this implementation, unchanged from the values specified in the pipeline design (Section A); each sample's `final_weight` is the product $w_{lexical} \times w_{semantic} \times w_{source} \times w_{trust}$, with $w_{trust} = 1.1$ only for `very_low` trust (RAG Chunk) and 1.0 otherwise.

Table 9: Source-aware feature weighting table (static; not yet grid-searched per cell, see Section A).

Input type	Lexical	Semantic	Source
RAG Chunk	0.80	1.40	1.30
User Prompt	1.30	1.00	1.00
PDF / DOCX / File	1.10	1.00	1.40
Email	1.00	1.00	1.20
URL	1.00	1.00	1.10
System Prompt	1.00	1.00	1.00

4.3.4 Dual-Stream Feature Extraction

Because xTRam1 (Direct injection) always has an empty `context`, extracting features only from the context stream would leave the model "blind" to direct-injection samples, whose attack signal lives entirely in `user_intent`. To address this, `extract_dataframe_features_dual_stream()` runs the full lexical/semantic/source pipeline *twice* per sample — once on the context stream (prefix `ctx_`, using `context_input_type`/`context_trust_level`) and once on the user-intent stream (prefix `intent_`, using `user_intent_input_type`/`user_intent_trust_level`) — and concatenates both feature blocks. Post-extraction validation confirmed non-zero TF-IDF signal on 86.80% of `ctx_*` rows versus 97.92% of `intent_*` rows, consistent with `user_intent` being populated in every sample while `context` is empty for all of xTRam1.

4.3.5 Chunk-Aware Aggregation

Since Step 2 may split long contexts into multiple overlapping chunks (Section 4.2), features are extracted per chunk and aggregated per sample: token/keyword/imperative/URL/email counts are summed, average token length is averaged, and all remaining scalar features (including the three attack-similarity scores) take the maximum across chunks — so that a payload confined to a single chunk is not diluted when only a small fraction of a long document

is malicious. Chunk-level embeddings are combined by mean-pooling. Across the full run, chunk-level feature extraction failed on 0/87,794 context chunks and 0/77,751 intent chunks (0.00% failure rate).

4.3.6 Output and Validation

Applying the full pipeline to all 77,314 samples produced a feature table of 950 columns (882 new features beyond the 68 input columns); after dropping non-numeric and metadata columns (`label`, `split`, ...), the final feature matrix has shape $77{,}314 \times 906$, with no NaN or Inf values. A consistency check comparing the matrix built manually against the one produced by the high-level `prepare_detection_input()` API showed a maximum absolute difference of 0.0, confirming reproducibility. This feature matrix, together with the aligned label and split arrays, is passed to Step 4 (Cascade Detection, Section 4.4).

4.4. Step 4: Cascade Detection Model

Step 4 consumes the feature matrix produced by Step 3 (`feature_df.parquet`, 77,314 rows × 906 features; 61,849 train / 7,730 validation) and implements the three-stage cascade of Fig. 1: a rule-based pre-filter, a lightweight DistilBERT classifier, and a full BERT classifier augmented with Mahalanobis out-of-distribution (OOD) detection. For every sample the cascade emits a label, a confidence score, an OOD flag, and a `path_taken` tag (`rule` / `distilbert` / `bert_ood`), the latter required for the per-branch sample sizes reported in the Ablation Study (Section 6.2). Training of the two neural components (Steps 4.2–4.3) was carried out in separate Kaggle GPU sessions (notebooks 04a and 04b); end-to-end cascade evaluation (Section 4.4.6) was run locally, CPU-only, from the saved checkpoints, without any further training.

4.4.1 Rule-based Pre-filter

The pre-filter applies three mechanisms in strict priority order (Algorithm 3), stopping at the first mechanism that reaches a decision: (1) an *exact-match blocklist*, mined in a data-driven way from the training split; (2) a small set of nine hand-written *regex* patterns for unambiguous injection phrasing (e.g., *"ignore previous instructions"*, *"you are now in developer mode"*); and (3) a *keyword-threshold* rule combining `keyword_density` and `suspicious_pattern_score` from Step 3.

For dual-stream inputs, the pre-filter is applied independently to the `context` and `user_intent` streams and combined with an OR rule for *attack* (either stream flagging is sufficient to block) and an AND rule for *benign* (both streams must independently agree before the row bypasses the model), mirroring the asymmetric-risk design of Step 3.

Algorithm 3 Rule-based Pre-filter (priority-ordered)

Require: text t (context or user_intent), lexical features f, blocklist B, thresholds τ_{kd}, τ_{susp}

1: **if** $t \in B_{\text{attack}}$ **then return** $(1, \text{conf} = 1.0, \text{“exact_match”})$
2: **end if**
3: **if** $t \in B_{\text{benign}}$ **then return** $(0, \text{conf} = 1.0, \text{“exact_match”})$
4: **end if**
5: **if** t matches any pattern in AttackRegexPatterns **then return** $(1, \text{conf} = 1.0, \text{“regex”})$
6: **end if**
7: **if** $f.\text{keyword_density} \geq \tau_{kd}$ **and** $f.\text{suspicious_score} \geq \tau_{susp}$ **then return** $(1, \text{conf} = 0.90, \text{“keyword”})$
8: **end if**
9: **return** $(\text{None}, 0, \text{“”})$ ▷ escalate to DistilBERT

Exact-match blocklist. Candidate sentences are mined on the training split only, requiring ≥ 5 exact repeats with 100% label purity. Following the source-aware protocol of Step 3, context-side mining uses the full training set (meaningful only for BIPIA, since xTRam1 has empty context), while user_intent-side mining is restricted to the Direct/xTRam1 subset — mining across all sources on user_intent would otherwise capture spurious correlations between a benign, frequently repeated question and a source that happens to carry a malicious context. Any sentence appearing in both the attack and benign sets is removed from both to avoid resolving the conflict arbitrarily. On this dataset the blocklist mining yielded **0 attack and 0 benign entries**: no sentence in either stream repeats verbatim ≥ 5 times with pure labels, so the exact-match mechanism contributes zero coverage here and is retained purely as a mechanism for future, more repetitive traffic (e.g., production logs), not as an active component in the present evaluation.

Regex pre-filter. Evaluated stand-alone on the validation split ($n = 7{,}730$): 23 rows matched (0.30% coverage), with **precision** = 1.0000 and zero false positives, confirming the patterns are conservative enough to fire only on unambiguous cases.

Keyword-threshold pre-filter. A grid search over $(\tau_{kd}, \tau_{susp}) \in [0.10, 0.90]^2$ (step 0.05) on the validation split required *both* precision ≥ 0.98 and a minimum reliable sample size $n \geq 30$ before a threshold pair would be trusted as an absolute (model-bypassing) decision. Of 289 grid cells, only 16 produced any match at all; **none** satisfied both constraints simultaneously (Table 10). The combination with the largest sample size ($\tau_{kd} = 0.10, \tau_{susp} = 0.10$, $n = 44$) reached only precision = 0.8182, while the one combination reaching precision = 1.0000 had only $n = 4$ — too small to trust as an unconditional gate. Consequently the keyword-threshold mechanism was deliberately disabled (thresh-

olds fixed at 1.01/1.01, outside the $[0, 1]$ feature range so the rule can never fire) rather than deployed with an unvalidated operating point; this is documented as an open item pending a larger and more diverse validation set (Section 9), not silently dropped.

Table 10: Keyword-threshold pre-filter grid search on validation (n = 7,730); top combinations by `n_match`. Required precision ≥ 0.98 and $n_match \geq 30$ jointly — no combination qualifies.

τ_{kd}	τ_{susp}	n_match	Precision
0.10	0.10	44	0.8182
0.15	0.10	36	0.8056
0.20	0.10	24	0.8333
0.25	0.10	18	0.7778
0.15	0.15	12	0.8333
0.20	0.15	11	0.8182
0.30	0.10	4	**1.0000**
0.10	0.20	3	0.3333

Combined pre-filter. With the blocklist empty and the keyword mechanism disabled, the pre-filter's effective coverage on validation reduces to the regex mechanism alone: 23/7,730 rows (0.30%) decided directly, precision 1.0000, and **99.70% of traffic escalated to DistilBERT**.

4.4.2 Lightweight Model: DistilBERT

Rows not resolved by the pre-filter are encoded as text pairs `(context, user_intent)` using `DistilBertTokenizerFast`, with `truncation="only_first"` to preserve `user_intent` in full and truncate only the (typically longer, less critical) context; a fallback to `truncation="longest_first"` handles the rare case where `context` is empty (all of xTRam1) and `user_intent` alone already exceeds `max_length`. A token-overflow check on the training split found that **655/61,849 rows (1.06%)** have a `user_intent` alone exceeding 384 tokens, motivating the chunk-aware inference wrapper described in Section 4.4.5.

Training uses Focal Loss ([7], $\gamma = 2.0$) with class weights computed directly from the training label distribution via `compute_class_weight("balanced")` rather than fixed constants, mixed-precision (`torch.amp`), gradient clipping (max-norm 1.0), and a linear warmup schedule (10% warmup steps). Table 11 reports the per-epoch validation metrics; the best checkpoint (epoch 3, selected by validation F1, *never* by test) is retained.

Full validation evaluation of the epoch-3 checkpoint (Table 12) shows an overall accuracy of 0.9724, with the injection class (label = 1) reaching precision 0.9762 / recall 0.9677 / F1 0.9719. Wall-clock training on a single Kaggle GPU took 37.8 minutes for 3 epochs (0.46 s/batch, 3,865 steps/epoch).

Table 11: DistilBERT fine-tuning (batch=16, lr=2×10^{-5}, max_length=384, $\gamma = 2.0$), validation metrics per epoch (n = 7,730).

Epoch	Train loss	Val F1	Val P	Val R
1	0.0505	0.9533	0.9688	0.9383
2	0.0225	0.9661	0.9495	0.9832
3	**0.0163**	**0.9719**	0.9762	0.9677

Table 12: DistilBERT standalone evaluation on validation, broken down by source (positive/injection class metrics; Accuracy is overall).

Group	n	Acc.	P	R	F1
Overall	7,730	0.9724	0.9762	0.9677	0.9719
BIPIA (indirect)	6,717	0.9693	0.9752	0.9657	0.9704
xTRam1 (direct)	1,013	0.9931	0.9870	0.9902	0.9886

4.4.3 Cascade Confidence Threshold

Following the protocol of Section A (no hardcoded thresholds), the DistilBERT-to-BERT hand-off threshold was chosen by grid search over $\theta \in [0.50, 0.98]$ (step 0.02) on the validation split, selecting the *smallest* θ for which the subset of validation rows with DistilBERT confidence $\geq \theta$ reaches accuracy ≥ 0.99 (Table 13). This selects $\theta = 0.66$ (coverage 94.83%, accuracy 0.9909).

Table 13: Cascade confidence threshold grid search on validation (n = 7,730); accuracy is computed on the subset with DistilBERT confidence $\geq \theta$.

θ	n (conf.$\geq \theta$)	Coverage	Accuracy
0.50	7,730	100.00%	0.9724
0.60	7,428	96.09%	0.9872
0.64	7,353	95.12%	0.9899
0.66	**7,330**	**94.83%**	**0.9909**
0.70	7,286	94.26%	0.9925
0.72	7,274	94.10%	0.9930
0.80	7,159	92.61%	0.9954
0.90	6,897	89.22%	0.9981
0.98	1,908	24.68%	1.0000

Reproducibility note. DistilBERT training (notebook 04a) and BERT training (notebook 04b) were executed as two independent Kaggle kernel sessions that do not share a working directory; 04b was unable to locate the `cascade_threshold.json` persisted by 04a and fell back to a hardcoded default of $\theta = 0.72$, which is what was ultimately saved to `cascade_config_v2.json` and used for the end-to-end evaluation in Section 4.4.6. As Table 13 shows, $\theta = 0.72$ still satisfies the accuracy ≥ 0.99 criterion (0.9930), so the deployed operating point is valid but more conservative (lower coverage, higher precision on the DistilBERT-only path) than the grid-search optimum of 0.66. This artifact-passing gap between training sessions is logged as a reproducibility

limitation in Section 9; the fix is to persist cascade configuration as a versioned dataset input rather than a per-session kernel output.

4.4.4 Full Model: BERT and Mahalanobis OOD Detection

Rows for which DistilBERT confidence falls below θ are routed to a fine-tuned `bert-base-uncased` classifier (batch=8, max_length=384 — increased from an earlier max_length=256 variant, bert_v1, to eliminate a max-length confound with DistilBERT in the Ablation Study), trained under the same Focal Loss / class-weight / mixed-precision protocol as DistilBERT (Table 14).

Table 14: BERT fine-tuning (batch=8, lr=2×10^{-5}, max_length=384, $\gamma = 2.0$), validation metrics per epoch ($n = 7{,}730$).

Epoch	Train loss	Val F1	Val P	Val R
1	0.0541	0.9613	0.9592	0.9635
2	0.0250	0.9722	0.9692	0.9753
3	**0.0176**	**0.9733**	0.9740	0.9727

Evaluated standalone on the full validation split (chunk-aware inference, Section 4.4.5), BERT reaches accuracy 0.9837 overall (Table 15) — marginally higher than DistilBERT's 0.9724 — at a modestly higher per-batch cost (0.51 s/batch vs. 0.46 s/batch, ∼1.11×) but roughly 2.3× the total wall-clock training time (86.8 vs. 37.8 minutes for 3 epochs), driven primarily by BERT's smaller batch size (8 vs. 16, hence 7,731 vs. 3,865 optimizer steps/epoch) rather than a proportionally heavier forward pass, motivating its restriction to the confidence-escalated subset only.

For OOD detection, class-conditional means and a shared (pooled) covariance are fit on the 768-dimensional `[CLS]` embeddings of the *training* split (61,849 × 768), following Lee et al. [8]. Because neither BIPIA nor xTRam1 contains labeled OOD samples, the OOD threshold is calibrated as the $(1 - \alpha)$-percentile of the Mahalanobis distance distribution on *in-distribution* validation embeddings, with target false-positive rate $\alpha = 5\%$ — a statistical anomaly threshold, not one validated against genuine OOD ground truth (Section 9). This yields OOD_THRESHOLD = 2,296.03, flagging 387/7,730 validation rows (5.01%), consistent with the target rate by construction. As a sanity check (not a calibration signal), the mean OOD score is markedly lower for correctly classified validation rows (763.43) than for misclassified rows (3,413.71), suggesting the statistic does correlate with BERT's own error propensity even without OOD ground truth.

4.4.5 Chunk-aware Inference

To avoid the truncation risk identified in Section 4.4.2 (long contexts silently dropping a mid- or end-of-text payload), both DistilBERT and BERT inference reuse `chunk_for_model()` from Step 2 (Algorithm 2) rather than the tokenizer's default truncation. For each row, the per-chunk context budget is computed dynamically as max_length − |tokens(user_intent)| − 3 (3 reserved for the `[CLS]` / `[SEP]` / `[SEP]` special tokens of a text-pair encoding); chunks are then classified independently and aggregated per row with an OR rule for the attack class (any chunk predicted attack ⇒ row is attack, taking the most confident attacking chunk's score) and, for BERT, the hidden state of the *decision-making* chunk is retained for OOD scoring. This aggregation choice is deliberately conservative (biased toward not missing a localized payload) at the cost of precision, which partially explains the lower precision observed on the escalated subset in Section 4.4.6. Because Mahalanobis parameters must be fit on the same hidden-state distribution seen at inference time, `fit_mahalanobis_chunk_aware()` refits class means and threshold on chunk-level (not whole-document) embeddings for both train and validation, avoiding a train/inference distribution mismatch that would otherwise miscalibrate the OOD threshold.

4.4.6 End-to-End Cascade Evaluation

The full three-stage cascade (pre-filter θ = regex-only, DistilBERT threshold $\theta = 0.72$, BERT+OOD, Section 4.4.3) was run on the entire validation split ($n = 7{,}730$). Table 16 reports the routing distribution and per-path metrics; 96.53% of traffic is resolved by the rule/DistilBERT path, leaving only 3.47% (268 rows) escalated to BERT+OOD — precisely the samples DistilBERT itself was least confident about.

Overall cascade accuracy is 0.9828 (precision 0.9689, recall 0.9971, F1 0.9828; confusion matrix TP=3,796, TN=3,801, FP=122, FN=11), broken down by source in Table 17. The escalated (`bert_ood`) subset is markedly harder than the rest of the traffic — its precision (0.3364) is far below the pipeline's overall precision, i.e., **most of the cascade's false positives are concentrated in the 3.47% of traffic that reaches the full model**, consistent with this subset being, by construction, exactly the cases DistilBERT itself could not resolve confidently.

On the escalated subset, the OOD flag correlates with BERT's own correctness even absent ground-truth OOD labels: of the 268 escalated rows, 30.97% are flagged OOD overall, but the flag rate is markedly higher among BERT's *incorrect* predictions (53.85%) than among its *correct* ones (21.58%) — supporting its intended use as a downstream escalation signal for Risk Scoring (Section 4.5), even though it was not designed as, and should not be read as, a calibrated attack detector in its own right.

Table 15: BERT standalone evaluation on validation, broken down by source (positive/injection class metrics; Accuracy is overall).

Group	n	Acc.	P	R	F1
Overall	7,730	0.9837	0.9706	0.9971	0.9837
BIPIA (indirect)	6,717	0.9820	0.9684	0.9980	0.9830
xTRam1 (direct)	1,013	0.9951	0.9967	0.9870	0.9918

Table 16: Cascade routing and per-path metrics on validation ($n = 7{,}730$; injection-class precision/recall/F1).

Path	n	%	Acc.	P	R	F1
rule (regex)	23	0.30%	1.0000	1.0000	1.0000	1.0000
distilbert	7,439	96.24%	0.9926	0.9871	0.9984	0.9927
bert_ood	268	3.47%	0.7090	0.3364	0.8810	0.4868

4.4.7 Cascade vs. DistilBERT-alone: Statistical Comparison

To isolate the actual contribution of the BERT+OOD stage, DistilBERT-alone (chunk-aware, no cascade routing) was also run on the full validation split, reaching accuracy 0.9842 — *marginally higher* than the full cascade's 0.9828 ($\Delta = -0.0014$). Restricting the comparison to the 268 escalated rows (the only rows where the two paths can actually disagree), DistilBERT reaches 0.7500 accuracy versus BERT's 0.7090 on the same subset (168 rows correct under both, 33 correct only under DistilBERT, 22 correct only under BERT, 45 wrong under both); McNemar's exact test on the 55 discordant pairs gives $p = 0.177$, i.e., **no statistically significant difference** between the two models on the hard subset at $\alpha = 0.05$.

This is a genuinely negative result for the cascade's second escalation stage on the current validation split and is reported as such rather than smoothed over: the architecture behaves exactly as designed operationally (routing the hardest 3.47% of traffic to the more expensive model), but on this data the extra model does not yield a measurable accuracy gain over simply trusting DistilBERT with a lower confidence bar. A supplementary power analysis on the true $n = 55$ discordant pairs (binomial test, $\alpha = 0.05$) shows the test is markedly underpowered: a true BERT win-rate of 65% would be detected with only 53.4% power, and 80% power is not reached until a true win-rate of 70% (power = 81.3%). *Note:* the power-analysis cell in notebook 04c contains a bug that re-derives its sample size from the full `bert_ood` path count ($n = 268$) rather than the actual McNemar discordant-pair count ($n = 55$), which would have overstated power (e.g., 90.0% at a 60% win-rate, versus the correct 24.8%); the figures above are recomputed directly from `scipy.stats.binom` at the correct $n = 55$ and should be used in place of the notebook's printed output. The corrected analysis reinforces, rather than weakens, the original conclusion: the $n = 55$ discordant pairs are too few to rule out even a moderate (65%) true advantage of the full model, and this should be revisited once a larger held-out hard subset is available (Section 9). This finding directly motivates the corresponding row of the Ablation Study (Section 6.2, "− OOD detection") and should be read alongside the Latency analysis (Section 6.5), since the BERT+OOD path is also the cascade's most computationally expensive branch.

4.4.8 Gate 3 Verification

The following criteria were checked before proceeding to Risk Scoring (Step 5, Section 4.5): (i) the test split was not accessed at any point in threshold selection or model checkpointing — all grid searches and checkpoint selection used validation F1/accuracy only; (ii) every threshold in this section (§4.4.1–§4.4.3) is justified by an explicit validation-set search rather than hardcoded, per the protocol of Section A, with the single documented exception being the threshold-persistence gap noted in Section 4.4.3; (iii) every table in this section reports its sample size n alongside point estimates, per the Data Splitting Protocol (Section 5.1); and (iv) `path_taken` is recorded for all 7,730 validation rows, enabling the per-component ablation in Section 6.2.

4.5. Step 5: Risk Scoring and Calibration

Step 5 consumes the cascade output of Step 4 (label, confidence, OOD flag, `path_taken`) together with anomaly-signal columns produced in Steps 1–3 and converts them into a bounded risk score and a discrete decision, following the formula of Section 3:

$$\text{Raw Score} = \text{Base} \times W_{\text{attack}} \times M_{\text{trust}} \tag{2}$$

$$\text{Adjusted Score} = \text{Raw} + B_{\text{anomaly}} + S_{\text{feature}} \tag{3}$$

$$\text{Final Score} = \text{clip}(\text{Adjusted} \times M_{\text{context}}, 0, 1) \tag{4}$$

All row-level computation is deterministic and rule-based (no additional model inference); dual-stream signals (context/user_intent) are merged before scoring using a "worse-of-the-two" rule for continuous risk-increasing signals (trust level, anomaly counts), consistent with the asymmetric OR/AND merging already used for the rule-based pre-filter (Section 4.4.1).

Table 17: Cascade evaluation broken down by source / attack type (validation, $n = 7{,}730$).

Group	n	Acc.	P	R	F1
BIPIA (indirect)	6,717	0.9811	0.9670	0.9977	0.9821
xTRam1 (direct)	1,013	0.9941	0.9902	0.9902	0.9902

4.5.1 Base Score: Confidence-to-Risk Conversion

The cascade's `confidence` output is $P(\hat{y})$, the probability of the *predicted* label, not $P(\text{attack})$. Using it directly as the Base Score would invert the risk ordering whenever the model confidently predicts *benign* (high confidence would then map to high risk). This is corrected as

$$\text{Base} = \begin{cases} \text{confidence}, & \hat{y} = 1 \text{ (attack)} \\ 1 - \text{confidence}, & \hat{y} = 0 \text{ (benign)} \end{cases} \tag{5}$$

so that confident-attack and confident-benign predictions map to opposite ends of the risk scale, as intended.

4.5.2 Attack Type Weight and Trust Level Multiplier

$W_{\text{attack}} \in \{0.6, 0.8, 1.0\}$ for Direct/Indirect/Data-Exfiltration respectively (Table 9-style mapping, mult. pipeline design). The current dataset schema (Section 5.1) only distinguishes Direct (xTRam1) and Indirect (BIPIA); no row carries a Data-Exfiltration label, so $W_{\text{attack}} = 1.0$ is retained in the implementation for spec completeness but is never exercised on this data — analogous to the RAG-chunk-only `very_low` branch already noted in Step 1. $M_{\text{trust}} \in \{1.0, 1.0, 1.2, 1.4\}$ for High/Medium/Low/Very-Low, applied to whichever of `context_trust_level`/`user_intent_trust_level` is *less* trustworthy for a given row.

4.5.3 Anomaly Bonus

The Anomaly Bonus accumulates additively (not mutually exclusive) over three signals carried from Steps 1–2: +0.10 for a hidden-text flag, +0.08 for a non-empty deobfuscation-technique list, and +0.08 for a positive delimiter-hit count. An audit across all three splits (Table 18) shows that the hidden-text and deobfuscation components are **structurally inactive on this dataset**: consistent with Table 5 (zero real hidden CSS styles and zero meaningfully decoded base64 payloads anywhere in BIPIA/xTRam1), *no row in train, validation, or test* ever triggers either bonus. Only the delimiter-hit bonus fires in practice, on 1,476/7,730 validation rows (19.1%).

4.5.4 Feature Signal: Bridging Source-aware Weighting into Risk Scoring

Prior to this revision, the Feature Signal term of Eq. 3 was a dead placeholder fixed at 0.0, meaning the source-aware feature weighting computed in Step 3 (Table 9) never actually reached the final decision when the cascade classifiers operate on raw text rather than the weighted feature vector (an issue explicitly surfaced during early Ablation Study runs, Section 6.2: disabling trust-weighting produced 0% measurable difference, because DistilBERT/BERT do not read the feature vector at all). `compute_feature_signal()` closes this gap by combining the (already source-aware-weighted) maximum lexical suspicious-pattern score and maximum semantic attack-similarity score across the two dual-stream branches into a single scalar in $[0, \text{scale}]$:

$$S_{\text{feature}} = \text{scale} \times \left(0.5 \cdot \hat{L} + 0.5 \cdot \hat{A}\right) \tag{6}$$

with $\hat{L}$ the (clipped, normalized) lexical component and $\hat{A}$ the (clipped) semantic attack-similarity component. A grid search over scale $\in [0.00, 0.50]$ (step 0.05) on validation, using the F1 of the Warn/Block decision boundary, found the criterion **monotonically decreasing** in scale, with the optimum at scale = 0.00 (F1= 0.9816, missed-attack rate 0.16%) versus scale = 0.50 (F1= 0.9787) — i.e., **the Feature Signal term contributes nothing beneficial on this validation split and is best left at zero.** This is reported as a second negative finding alongside the OOD-detection result of Section 4.4.7, both of which motivate dedicated rows in the Ablation Study (Section 6.2).

4.5.5 Context Multiplier

M_{context} (tool-access ×1.2, sensitive-data ×1.3) requires runtime agent context not present in either offline benchmark and is therefore implemented as a no-op ($M_{\text{context}} \equiv 1.0$) for all experiments in this paper, consistent with its status as a proposed, not evaluated, component of the Response Layer (Section 7).

4.5.6 Risk Level Threshold Calibration

Two calibration passes were run on the validation split, reflecting two complementary objectives.

Pass 1 — F1-optimal boundary. A grid search over (low_max, medium_max, high_max) scored each candidate by the precision/recall/F1 of predicting

Table 18: Audit of Anomaly Bonus trigger rates across splits. Hidden-text and deobfuscation components never fire on this dataset (cf. Table 5).

Split	n	hidden_text > 0	techniques non-empty
Train	61,849	0	0
Validation	7,730	0	0
Test	7,735	0	0

"attack" at the Critical boundary (final_score > high_max). The optimum found was high_max = 0.75 (F1= 0.9441, precision= 0.9756, recall= 0.9146); several (low_max, medium_max) pairs tied at this F1 (Table 19), since the two lower boundaries do not affect this particular criterion and were therefore fixed separately in Pass 2.

Table 19: Sample of tied optima from the Pass-1 grid search (all share high_max = 0.75, F1= 0.9441).

low_max	medium_max	high_max
0.15	0.45	0.75
0.15	0.50	0.75
0.15	0.65	0.75
0.20	0.45	0.75
0.40	0.55	0.75

Pass 2 — safety-constrained calibration. Given the degeneracy of Pass 1 and the operational cost of a missed attack under the "Allow" decision (no further monitoring at all, Table 20), low_max was instead chosen as the *largest* value keeping the missed-attack rate on the Low/Allow bucket at or below a 1% safety budget, scanning $[0.05, 0.50)$: this yields low_max = 0.49 (empirical missed-attack rate 0.32%). high_max = 0.75 was retained from Pass 1, and medium_max = 0.62 was set at the midpoint between the two, with a monotonicity sanity check on the empirical attack rate of the Medium and High buckets.

This sanity check failed: the Medium bucket ($0.49 < \text{score} \leq 0.62$) has an empirical attack rate of 87.06%, *higher* than the High bucket ($0.62 < \text{score} \leq 0.75$) at 40.48% — the opposite of the monotonically increasing risk ordering the four-level scheme is intended to express. The notebook's own instrumentation flags this explicitly at run time but the pipeline does not currently block on it. We report this as an **unresolved calibration anomaly** rather than silently accepting the thresholds: a plausible cause is that Pass 1's F1 criterion (Critical-only, per the note above) selects a high_max that is not actually the boundary that best separates "High" from "Critical" risk, since it was never evaluated against that specific boundary. Until this is corrected (Section 9), the High/Warn tier's risk ordering relative to Medium/Monitor should not be treated as validated, even though the Low/Allow safety constraint and the overall Critical/Block precision are.

The final calibrated thresholds (low_max = 0.49, medium_max = 0.62, high_max = 0.75) were nonetheless carried forward to produce the operating decision distribution in Table 20, since the primary safety property (bounded missed-attack rate on Allow) does hold; the Medium/High ordering anomaly is treated as a follow-up item rather than a blocker for the rest of the pipeline, given that only $340 + 42 = 382$ rows (4.9% of validation) fall into these two intermediate tiers combined.

Table 20: Final risk-level / decision distribution on validation (n = 7,730) under the safety-constrained thresholds. Missed-attack rate within Allow = 0.32% (target ≤ 1%).

Risk Level	Decision	n (%)
Low	Allow	3,779 (48.9%)
Medium	Monitor	340 (4.4%)
High	Warn	42 (0.5%)
Critical	Block	3,569 (46.2%)

4.5.7 Gate 4 Verification

Test set access remains untouched throughout Step 5 — both calibration passes and the Feature-Signal-scale grid search use validation only, consistent with the Data Splitting Protocol (Section 5.1). Every weight and threshold in the risk-scoring formula is now empirically grounded rather than hardcoded, with the single exception of the OOD Escalation Bonus (+0.15, Section 4.4.4), which remains an initial value pending its own grid search and is logged as an open item in the Hyperparameter Appendix (Section A). With the safety constraint on the Low/Allow bucket verified (missed-attack rate 0.32% ≤ 1% target) and the Critical/Block boundary retaining its F1-optimal value, Gate 4 is passed and the pipeline proceeds to the offline evaluation harness (Section 5).

5. Experimental Setup

5.1. Datasets and Data Splitting Protocol

Two public datasets were used to train and evaluate the pipeline: BIPIA [2], representing *indirect prompt injection* (five scenarios: Email, Web QA, Table QA, Summarization, Code QA), and xTRam1/safe-guard-

prompt-injection [6], representing *direct prompt injection*. Both sources were mapped to a common schema of seven fields: `context` (external/retrieved content), `user_intent` (the user-typed request), `label` (0: benign, 1: injection), `source`, `attack_type` (Direct/Indirect), `input_type`, and `origin`.

5.1.1 Data Cleaning

Before merging, each source was independently deduplicated: BIPIA on the (`context`, `user_intent`) pair, and xTRam1 on the raw `text` field. Table 21 summarizes the cleaning results.

The `context` field of xTRam1 is inherently empty, since this dataset consists purely of *direct injection* samples originating from `user_intent`. All 10,137 such rows were assigned an empty `context`, which cleanly separates them from BIPIA's indirect-injection mechanism, where the attack signal resides in the retrieved `context`.

5.1.2 Stratified Data Splitting Protocol

The full set of 77,314 cleaned samples was split into an 80/10/10 (train/validation/test) partition, stratified jointly on `source`, `attack_type`, and `label` (Algorithm 4), using a fixed random seed (`seed=2026`). The split was performed exactly once, at the data-loading stage, and reused consistently across Steps 1–5, in accordance with the Data Splitting Protocol described in Section 4.4. The test set (10%) was locked immediately after this step and is used only once, for final reporting.

Algorithm 4 Stratified Data Splitting (`make_split`)

Require: DataFrame D with columns `source`, `attack_type`, `label`; seed s

1: D.`stratify_col` $\leftarrow$ `source` $\|$ `attack_type` $\|$ `label`
2: $D \leftarrow \text{Shuffle}(D, \text{random_state} = s)$
3: **for all** group G by `stratify_col` in D **do**
4: $n \leftarrow |G|$; $n_{\text{train}} \leftarrow \lfloor 0.8n \rfloor$; $n_{\text{val}} \leftarrow \lfloor 0.1n \rfloor$
5: Assign `split` = `"train"` to the first n_{train} rows of G
6: Assign `split` = `"val"` to the next n_{val} rows of G
7: Assign `split` = `"test"` to the remaining rows of G
8: **end for**
9: **return** D with the assigned `split` column

Table 22 shows the detailed distribution across `split` × `source` × `attack_type` × `label`, confirmed by the `validate_split()` check (Gate 0): the realized split ratios were 79.997%/9.998%/10.005% for train/val/test, negligibly different from the 80/10/10 target.

The final dataset is close to class-balanced overall (50.8% benign vs. 49.2% injection); the class-imbalance handling introduced at Step 4 (Focal Loss, class weighting) therefore mainly targets *within-source* imbalance (e.g., xTRam1 Direct skews toward benign, 5,652 vs. 2,456 injection samples in the training split) rather than global imbalance.

5.2. Evaluation Harness and Metrics

To guarantee that every number reported in Sections 6.1–6.5 uses an identical, reproducible definition, all downstream evaluation notebooks (07–11) share a single metrics module (`evaluation.py`) rather than re-implementing metric code per notebook; the module performs no training or prediction of its own and consumes only pre-computed `y_true`/`y_pred`/`y_score` arrays. All resampling-based procedures use a fixed seed (`RANDOM_SEED=2026`) for reproducibility.

5.2.1 Reporting Convention: Sample Size and Bootstrap Confidence Intervals

`report_with_n()` computes precision, recall, F1, and accuracy (`zero_division=0`) and always returns the sample size n alongside the point estimates, formalizing the convention already followed throughout Sections 4.1–4.5. When $n < 100$ (`MIN_N_FOR_NO_CI`), a 95% bootstrap confidence interval is attached automatically, using $B = 1{,}000$ percentile-method resamples (`bootstrap_ci()`) rather than left as an optional step; resamples that collapse to a single class for a given metric are skipped rather than crashing the estimator. This directly addresses the small-sample caveat already flagged in the cascade comparison of Section 4.4.7 (only 55 discordant pairs) and is required by design for the Adaptive Attack evaluation (Section 6.4), where individual attack-technique subsets are frequently small.

5.2.2 AUROC, False Positive Rate, and Attack Success Rate

`compute_auroc()` wraps `roc_auc_score` on continuous confidence scores, returning `NaN` rather than raising when a subset contains a single class (e.g., an all-benign or all-attack group), which would otherwise abort per-group aggregation. `compute_fpr()` restricts the false-positive-rate calculation to genuinely benign rows ($y_{\text{true}} = 0$) as FPR $= FP/(FP + TN)$, and `fpr_by_group()` applies this per `input_type` (email/RAG/URL/file/user prompt), feeding Section 6.3 directly. `compute_asr()` defines Attack Success Rate, for a batch of genuine attack samples subjected to an adversarial transformation, as the fraction subsequently predicted benign (i.e., evading detection); `compute_asr_with_ci()` pairs this with the same bootstrap procedure above, used throughout Section 6.4.

Table 21: Deduplication results for the two data sources.

Source	Raw rows	Duplicates removed	Clean rows
BIPIA	70,000	2,823	67,177
xTRam1	10,296	159	10,137
Total	80,296	2,982	**77,314**

Table 22: Detailed data distribution by split, source, attack type, and label.

Split	Source (attack_type)	Benign (0)	Injection (1)	Total	%
Train	BIPIA (Indirect)	25,741	28,000	53,741	80.0
	xTRam1 (Direct)	5,652	2,456	8,108	
Validation	BIPIA (Indirect)	3,217	3,500	6,717	10.0
	xTRam1 (Direct)	706	307	1,013	
Test	BIPIA (Indirect)	3,219	3,500	6,719	10.0
	xTRam1 (Direct)	708	308	1,016	
Total		**39,243**	**38,071**	**77,314**	100.0

5.2.3 Ablation Comparison

`compare_ablation()` generalizes the McNemar's exact test procedure already applied ad hoc to the cascade-vs-DistilBERT comparison in Section 4.4.7 (binomial test on discordant pairs, $p = 0.5$) into a reusable function that reports, for a baseline and an ablated prediction column, the F1 gap and whether the discordant-pair test reaches significance at $\alpha = 0.05$; `run_ablation_study()` applies this across an arbitrary set of ablated components in a single pass, producing the per-component table used in Section 6.2.

6. Results and Discussion

6.1. Baseline Comparison

Comparison was restricted to two benchmarks for which a compatible, license-available implementation could be run end-to-end: the in-distribution test split locked in Section 5.1 (BIPIA+xTRam1, $n = 7{,}735$) and an out-of-distribution (OOD) set synthesized from Open-Prompt-Injection [1] using its `combine` attack strategy, with SST2 sentiment classification as the target task and SMS-spam detection as the injection task ($n = 600$: 300 benign target prompts, 300 attacked). Consistent with the notebook's own scope note, AgentDojo (an agent-execution framework rather than a binary-labeled dataset) and PINT (a proprietary Lakera benchmark unavailable for offline evaluation) were out of scope, and PromptArmor was excluded because it requires GPT-4o API access not available to this study; these three are logged as limitations (Section 9) rather than omitted silently.

6.1.1 In-distribution Comparison

Table 23 compares the proposed cascade against three groups of baselines on the locked test split. **DataSentinel** [5] used the authors' released Mistral-7B-based fine-tuned checkpoint; because inference cost (≈4.7 s/sample) made the full 7,735-row test set impractical (≈10 h wall-clock), it was evaluated on a random $n = 300$ subsample (`random_state=2026`), which is reported explicitly per the sample-size convention of Section 5.2 rather than treated as equivalent to a full-test-set number.

On in-distribution data, the four transformer-scale methods (SVM, DistilBERT-alone, BERT-alone, Cascade) all cluster tightly in F1 (0.9828–0.9855); notably, DistilBERT-alone alone is marginally *higher* than the full Cascade (0.9855 vs. 0.9834), reinforcing the negative finding already reported for the BERT+OOD escalation stage in Section 4.4.7 rather than contradicting it. A purely lexical TF-IDF+SVM baseline is also highly competitive (F1= 0.9835) on this in-distribution data, though this should be weighed against its OOD behavior below. The two general-purpose open guard models and DataSentinel substantially underperform every task-specific fine-tuned method: PromptGuard v1 reaches only F1= 0.6665 (precision 0.5214, i.e., roughly one in two of its positive flags is a false alarm on this benchmark), PromptGuard v2 shows an extreme precision/recall imbalance (P= 0.9943, R= 0.0460) consistent with a decision threshold poorly calibrated for BIPIA/xTRam1-style inputs rather than an inherent inability to detect the attacks, and DataSentinel's subsampled accuracy (0.5567) is close to chance despite a high recall (0.9481), again driven by low precision (0.5387). These results are consistent with all three external systems being tuned primarily for direct/jailbreak-style prompts rather than the context-embedded indirect injection that dominates BIPIA.

Table 23: In-distribution comparison on the locked test split (BIPIA+xTRam1, $n = 7{,}735$ unless noted).

Method	P	R	F1	Acc.
TF-IDF + Logistic Regression	0.9372	0.9874	0.9616	0.9612
TF-IDF + Linear SVM	0.9706	0.9968	0.9835	0.9836
DistilBERT (standalone)	0.9734	0.9979	0.9855	0.9855
BERT (standalone)	0.9679	0.9982	0.9828	0.9828
PromptGuard v1 (86M)	0.5214	0.9236	0.6665	0.5451
PromptGuard v2 (86M)	0.9943	0.0460	0.0879	0.5302
DataSentinel ($n = 300$)	0.5387	0.9481	0.6871	0.5567
Cascade (Ours)	**0.9694**	**0.9979**	**0.9834**	**0.9835**

6.1.2 Out-of-distribution Comparison

Table 24 reports the same methods (excluding DataSentinel, not run on this benchmark) on the OOD set. PromptGuard was not evaluated here because of a notebook variable-scope issue — the malicious-label ID list resolved in the in-distribution cell was not carried into the OOD cell — and is logged as a reproducibility gap (Section 9) rather than a silent omission, in the same spirit as the threshold-persistence gap already disclosed in Section 4.4.3.

The OOD comparison is the more informative test of generalization. Both TF-IDF baselines collapse on recall (0.31–0.49) despite retaining high precision, confirming that a purely lexical decision boundary fitted on BIPIA/xTRam1 vocabulary does not transfer to the unrelated sentiment/spam domain of Open-Prompt-Injection. DistilBERT-alone generalizes best of all five methods (F1= 0.9868), again modestly exceeding the full Cascade (F1= 0.9788) — a second, independent data point (beyond Section 4.4.7) suggesting the BERT+OOD escalation stage is not currently adding measurable value over trusting DistilBERT directly. BERT-alone shows the largest in-distribution-to-OOD gap of any method (F1 falling from 0.9828 to 0.8100), the opposite direction from what its extra capacity might suggest. The Cascade nonetheless achieves perfect recall on this OOD set ($R = 1.0000$, no missed attacks) at a moderate precision cost (P= 0.9585), a safety-favoring profile consistent with the asymmetric-risk design already motivated in Section 4.4.1. Cascade routing also behaves very differently here than in-distribution: the rule-based pre-filter alone resolves 300/600 rows (50.00%) directly, versus only 27/7,735 (0.35%) on the locked test set, and of the 13 rows escalated all the way to BERT+OOD, all 13 (100%) are flagged by the Mahalanobis OOD detector — direct evidence that the detector correctly recognizes genuinely out-of-distribution inputs when it reaches them, though the small escalated count ($n = 13$) limits how strongly this single observation can be generalized.

6.2. Ablation Study

Ablation was measured on the held-out test split ($n = 7{,}735$, opened only after Gate 4, Section 4.5.6), with a sample counted as a positive (attack) prediction if its final decision is `Warn` or `Block` under the risk-level mapping of Table 20 — i.e., anything above the Medium/Monitor tier — rather than a coarser Block-only or externally imposed split. Four components were ablated against the same full-pipeline baseline, following the `compare_ablation()`/`run_ablation_study()` protocol of Section 5.2 (McNemar's exact test on discordant pairs, $\alpha = 0.05$): (B) Trust-weighting, by recomputing the Risk Scoring feature signal (Eq. 3) from the *unweighted* lexical/semantic components (dividing out the source-aware weights of Table 9) while reusing the same cascade predictions; (C) OOD detection, by disabling the OOD Escalation Bonus at the Risk Scoring stage while reusing the same cascade predictions and OOD flags; (D) Dual-stream, by re-running the full cascade on the neutralized `sanitization_stream` in place of the `analysis_stream` (Section 4.2), so that base64/delimiter/role-switch signals are redacted before detection rather than only before an eventual LLM call; and (E) Rule-prefilter, by removing Stage 1 entirely so every row is routed directly to DistilBERT, re-implemented by re-composing the public Stage 2/3 functions rather than by retraining anything.

Removing **Trust-weighting** produces an exactly identical prediction set (0 discordant pairs, ΔF1= 0.0000). This is mechanically expected rather than a new finding: Section 4.5 already established that the Feature-Signal-scale grid search on validation selects scale = 0.00 as optimal, meaning the calibrated pipeline already routes zero contribution from source-aware weighting into the final risk score at this stage. This ablation reproduces that result on the independent test set; it does not indicate that source-aware weighting is unhelpful inside the Step 3 feature extraction itself (Section 4.3) — only that its current bridge into Risk Scoring is inactive, consistent with the negative finding already reported.

Removing **OOD detection** very slightly *improves* F1 (0.9824 → 0.9828) and AUROC (0.9924 → 0.9943), but with only 3 discordant pairs the test cannot reject the null of no difference ($p = 0.250$). The direction of this (non-significant) effect is at least consistent with, though not strong independent confirmation of, the negative result already reported for the BERT+OOD escalation stage in Section 4.4.7.

Table 24: Out-of-distribution comparison (Open-Prompt-Injection, `combine` attack, SST2/SMS-spam, $n = 600$).

Method	P	R	F1	Acc.
TF-IDF + Logistic Regression	0.9542	0.4867	0.6446	0.7317
TF-IDF + Linear SVM	0.9300	0.3100	0.4650	0.6433
DistilBERT (standalone)	0.9803	0.9933	0.9868	0.9867
BERT (standalone)	0.6936	0.9733	0.8100	0.7717
Cascade (Ours)	**0.9585**	**1.0000**	**0.9788**	**0.9783**

Table 25: Ablation study on the held-out test set ($n =$ 7,735); positive = decision $\in$ {Warn, Block}.

Configuration	P	R	F1	AUROC
Full pipeline	0.9679	0.9974	0.9824	0.9924
− Trust-weighting	0.9679	0.9974	0.9824	0.9924
− OOD detection	0.9686	0.9974	0.9828	0.9943
− Dual-stream	0.9669	0.9963	0.9814	0.9918
− Rule-prefilter	0.9679	0.9974	0.9824	0.9924

Removing **Dual-stream** is the only ablation reaching statistical significance ($p = 0.008$, $n = 8$ discordant pairs): F1 drops from 0.9824 to 0.9814 and AUROC from 0.9924 to 0.9918. The effect size is small in absolute terms, but it is the sole component for which the paper can make a statistically defensible claim of a genuine, if modest, accuracy contribution — consistent with dual-stream's designed role (Section 4.2, Section 4.3) of preserving attack signal that sanitization would otherwise redact, which matters most for direct-injection (xTRam1) rows where `context` is empty and the entire signal lives in `user_intent`.

Removing the **Rule-prefilter** again produces zero discordant pairs. This is consistent with the regex mechanism's already-reported perfect precision (1.0000, Section 4.4.1) on the rows it fires on (27/7,735, 0.35%): without it, those 27 rows are simply routed to DistilBERT instead, which classifies them identically (Stage-2 confident count rises from 7,418/7,708 to 7,445/7,735, and the Stage-3 BERT+OOD subset is unchanged at 290 rows with 81 OOD flags in both configurations). On this dataset the rule-prefilter therefore currently functions as a latency optimization (bypassing model inference on unambiguous cases) rather than an accuracy contributor, mirroring the already-disclosed inactivity of its exact-match blocklist sub-component (Section 4.4.1).

Taken together, only 1 of 4 ablated components reaches statistical significance, and it is the one component (Dual-stream) whose contribution was not already flagged as structurally disabled at an earlier gate — Trust-weighting's feature-signal bridge (Section 4.5) and the Rule-prefilter's blocklist (Section 4.4.1) were both already reported inactive on this dataset, so their null ablation effect here is a consistency check across sections rather than a new result.

6.3. False Positive Rate on Clean Traffic

FPR was measured on all benign (label = 0) rows of the held-out test split ($n =$ 3,927), running the full pipeline (cascade + Risk Scoring, same `BEST_FEATURE_SIGNAL_SCALE`= 0.00 as calibrated in Section 4.5) and counting a decision of `Warn` or `Block` as a false positive. Following the protocol requirement to disaggregate by source (rather than report one pooled number that can mask large per-source variation), Table 27 reports FPR separately by `context_input_type`; per the Gate-5 convention already used elsewhere (Section 5.2), groups with $n < 100$ receive an automatic bootstrap 95% CI. No `system_prompt` rows appear in this benign subset, consistent with BIPIA/xTRam1 not containing any system-prompt-sourced samples (already noted for the analogous `very_low`/Data-Exfiltration gaps in Sections 4.1 and 4.5).

The pooled FPR (3.21%) conceals more than a 7-fold spread across sources, from 0.00% on benign user prompts ($n =$ 708; the pipeline makes zero false-positive decisions on this, typically the most central traffic type in an LLM application) up to 7.61% on benign URLs ($n =$ 92, 95% CI [2.17, 13.04]). The two highest-FPR groups, URL and RAG chunk, are also the two lowest-trust categories (Low and Very Low respectively, Table 1) and carry the largest trust-level risk multiplier M_{trust} (1.2 and 1.4 respectively, Section 4.5); however, Email and File nominally share the same Low trust level as URL yet show substantially lower FPR (1.53% and 1.16%), so trust level alone does not fully explain the pattern. Since the source-aware feature weights of Table 9 were already confirmed inactive at the calibrated Feature-Signal scale of 0.00 (Section 4.5), the residual File/Email/URL variation most plausibly reflects surface-level lexical properties that the cascade's shared (not source-conditioned) text features respond to regardless of declared source — URL strings in particular contain slashes, query-parameter delimiters, and encoded characters that may resemble the delimiter- and pattern-based signals used elsewhere in the pipeline (Section 4.3) — rather than any explicit per-source calibration difference; this is reported as a hypothesis rather than a confirmed mechanism and is flagged for dedicated error analysis in Future Work (Section 10). Practically, the result indicates that a deployment weighting URL-sourced content heavily (e.g., a browsing agent) should expect a meaningfully higher

Table 26: Statistical significance of each ablation vs. the full pipeline (McNemar's exact test on discordant pairs).

Component removed	n **discordant**	**ΔF1**	p**-value**	**Sig.** (α=0.05)
Trust-weighting	0	0.0000	–	No
OOD detection	3	−0.0004	0.250	No
Dual-stream	8	0.0010	0.008	**Yes**
Rule-prefilter	0	0.0000	–	No

Table 27: False Positive Rate on clean (benign) test-split traffic, by input source (n = 3,927 total).

Input type (benign)	n	**FPR**	**95% CI**
URL / web content	92	0.0761	[0.0217, 0.1304]
RAG chunk	2,196	0.0487	—
Email	326	0.0153	—
File (PDF/DOCX)	605	0.0116	—
User prompt	708	0.0000	—
Overall (pooled, reference only)	3,927	**0.0321**	—

nuisance-alert rate than one dominated by direct user prompts, even though both are measured under the same calibrated thresholds.

6.4. Adaptive / Red-Team Attack Evaluation

Robustness was probed on a fixed random subsample of $n = 50$ genuine attack rows from the held-out test split (`random_state=2026`), used consistently across every technique and both attack modes so that results are directly comparable; per the reporting convention of Section 5.2, n = 50 <`MIN_N_FOR_NO_CI` triggers an automatic 95% bootstrap CI on every Attack Success Rate (ASR) below. On this subset, unmodified (non-attacked) inputs are detected perfectly (baseline ASR = 0.00%; all 50 rows resolved confidently by DistilBERT at Stage 2, none escalated), consistent with the near-ceiling recall already reported on the full validation cascade (Section 4.4.6).

Two attack modes were evaluated per technique (paraphrase via synonym substitution, homoglyph substitution, genuine MarianMT-based en→vi→en round-trip translation, and nested/delimiter payload wrapping): a **static, black-box** attack (single transformation, fixed intensity = 0.4, no knowledge of the pipeline), and an **adaptive, white-box** attack in which the attacker is assumed to know the deployed cascade and OOD thresholds (θ = 0.72, OOD= 2,296.03) and iteratively re-queries the real pipeline up to 4 rounds (intensity schedule 0.4 → 0.6 → 0.8 → 1.0), stacking additional nested-wrapping on top of the primary technique from round 2 onward (except for the `nested` technique itself, to keep the comparison fair) and stopping early per-sample once evasion is achieved.

Under the static, black-box condition, only translation round-trip achieves substantial evasion (ASR= 80.00%, 95% CI [68, 90]); homoglyph substitution is moderate (28.00%, CI [16, 40]), while paraphrase and nested-payload wrapping are both essentially defeated (0.00%). This ordering is consistent with the pipeline's design: nested/delimiter wrapping is exactly the anomaly class the preprocessing and feature-extraction stages target directly (hidden-text and delimiter signals, Sections 4.2–4.3), so its near-zero static ASR is an expected consequence rather than a new finding, whereas a genuine machine-translation round-trip rewrites surface form completely and is not directly targeted by any pipeline component.

Table 28: Static (black-box) vs. adaptive (white-box) Attack Success Rate per technique (n = 50 attack samples).

Technique	**Static ASR**	**Adaptive ASR**	
Paraphrase	0.00%	0.00%	+0.
Homoglyph	28.00%	84.00%	+56.
Translation round-trip	80.00%	94.00%	+14.
Nested payload	0.00%	2.00%	+2.

The adaptive condition changes this picture substantially. Homoglyph ASR rises from 28.00% to 84.00% (+56 pp, the largest gap of the four techniques and highly significant, $p < 0.0001$), and translation rises further from an already-high 80.00% to 94.00% (p = 0.0156); both evasions that succeed do so quickly (average 2.0 and 1.3 iterations respectively, out of a maximum of 4), indicating the vulnerability is reached within the first one or two rounds of increased intensity rather than requiring exhaustive search. Paraphrase remains at exactly 0.00% ASR under adaptation (zero discordant pairs — a synonym-substitution attacker gains nothing from knowing the pipeline's thresholds or increasing intensity), and nested payload rises only marginally to 2.00% and is not statistically significant (p = 1.0000). The practical implication is that a technique with negligible black-box risk (homoglyph, 28%) becomes the pipeline's single largest attack surface once

Table 29: Statistical significance of the static-vs-adaptive gap (McNemar's exact test, paired predictions, $\alpha = 0.05$) and, for adaptive attacks that evaded detection, the average iteration at which evasion occurred.

Technique	p-value	Sig.	Avg. iters (evaded)
Paraphrase	–	N/A (0 discordant)	n=0 (none evaded)
Homoglyph	< 0.0001	**Yes**	2.0 (n=42)
Translation round-trip	0.0156	**Yes**	1.3 (n=47)
Nested payload	1.0000	No	4.0 (n=1)

white-box query access is available (84%) — a genuine adaptive-robustness gap that is invisible in the static benchmark numbers reported in Sections 4.4.6 and 6.1, and is exactly the failure mode this evaluation was designed to surface.

6.4.1 Cross-lingual Generalization Probe

A separate probe measured a distinct failure mode: not an attacker adaptively evading detection, but the cascade's natural miss rate on genuinely different input language. Fifty attack and fifty benign test-split rows were translated one-way into Vietnamese (MarianMT `en-vi`, no round-trip back to English) and passed through the unmodified cascade. Recall on the Vietnamese attack text collapsed to 4.00% (95% CI [0.00, 10.00]) — a 96-percentage-point drop from the $\sim 100\%$ recall the same cascade achieves on unmodified English attack text on the equivalently sized baseline subset above — while the false-positive rate on Vietnamese benign text remained comparatively low (6.00%, 95% CI [0.00, 14.00]). The failure is therefore concentrated almost entirely in missed attacks rather than a general breakdown of discrimination, consistent with every trainable component of the cascade (DistilBERT/BERT fine-tuning, TF-IDF vocabulary, hand-written attack-keyword and regex lists of Sections 4.1–4.4) being fit exclusively on English-language BIPIA/xTRam1 text. This is reported as a language-generalization limitation distinct from adversarial robustness, and is carried forward into Limitations (Section 9) and Future Work (Section 10) rather than folded into the adaptive-attack ASR numbers above.

6.5. Latency and Overhead

Latency was measured single-request (batch size = 1, sequential, no concurrent load) on a Tesla T4 GPU (15.6 GB) with 33.7 GB RAM, using 5 warm-up calls per configuration (excluded from timing) followed by `torch.cuda.synchronize()`-bracketed wall-clock measurement per request, on $n = 200$ randomly sampled rows per configuration (`random_state=2026`). This measures per-request latency under a single-worker deployment rather than throughput under concurrent load, which is left to Future Work (Section 10).

The real per-stage routing distribution was measured directly on the full test split ($n = 7{,}735$) rather than assumed: `rule_regex` resolves 27 rows (0.35%), `distilbert` resolves 7,418 rows (95.90%), and `bert_ood` processes the remaining 290 rows (3.75%) — matching the routing already reported in Section 4.4.6. Grouping the first two into a combined *fast path* (rule and/or DistilBERT resolved, 96.25% of traffic) and the third into a *slow path* (escalated to BERT+OOD, 3.75%) gives the actual traffic split used to weight the system-wide average below, superseding the illustrative 80/20 split used for planning purposes in the pipeline design document.

Weighting the fast- and slow-path latency distributions by the real traffic split above gives a system-wide cascade p50 of 14.84 ms and p95 of 42.05 ms, a 1.91× speedup over the full-model baseline at p50 (28.38 ms) and throughput essentially double (49.47 vs. 26.07 req/s) — directly attributable to the 96.25% of traffic that never needs a BERT forward pass. This confirms the cascade design's latency motivation (Section 4.4) empirically rather than only architecturally.

The slow path itself, however, is *not* faster than the baseline per request — at 82.44 ms p50 it is nearly 3× slower than running BERT alone (28.38 ms), since an escalated row pays sequentially for a DistilBERT forward pass (Stage 2, which must run and be found low-confidence before escalation), a BERT forward pass (Stage 3), and Mahalanobis OOD scoring, rather than the cascade substituting one model for another on this subset. The system-wide gain therefore comes entirely from the low escalation rate (3.75%) rather than from any per-request efficiency on the escalated tail; a workload with a substantially higher OOD/hard-sample rate than this dataset's test split would erode the cascade's latency advantage accordingly, which is a relevant caveat for the Limitations discussion (Section 9).

7. Proposed Response Layer (System Design)

This section describes the architecture of the Response Layer that would sit downstream of Risk Scoring (Step 5, Section 4.5) in a deployed system. Consistent with the scope stated in Section 1, **this layer is not experimentally evaluated**: no alert is actually dispatched to Telegram or email, no IP address is actually blocked, and no attacker profile is built from live traffic. Instead, each component is exercised as an offline, in-memory simulation (`response_simulation.py`) driven by the

Table 30: Per-request latency and throughput, Tesla T4, batch size 1 ($n = 200$ per row unless noted).

Configuration	n	p50 (ms)	p95 (ms)	Thr. (req/s)
Baseline (full BERT, no cascade)	200	28.38	80.39	26.07
Cascade fast path (rule+DistilBERT, 96.25%)	200	14.79	28.58	56.31
Cascade slow path (+BERT+OOD, 3.75%)	200	82.44	123.73	12.01
Cascade, system-wide (weighted)	7,735	**14.84**	**42.05**	**49.47**

risk-scoring output of the validation split ($n = 7{,}730$), so that its intended logic can be demonstrated and its structural limitations made explicit before any production commitment.

7.1. Decision Mapping

The four-level decision mapping (Low→Allow, Medium→Monitor, High→Warn, Critical→Block) is re-implemented independently in this layer (`map_risk_to_decision()`) rather than simply reused from Step 5, as a consistency check. Applied to all 7,730 validation rows, the two decisions agree exactly (100% match against the `decision` column produced in Section 4.5), confirming that the mapping in Table 20 is deterministic and reproducible across pipeline stages. Of the layer's four sub-components, decision mapping is the only one we consider production-ready as implemented; the remaining three (Sections 7.2–7.4) are proposed designs.

7.2. Alert Simulation

`AlertSimulator` constructs a structured alert record (request ID, timestamp, source IP, user agent, risk level/score, attack type, and supporting evidence) for every row with decision Warn or Block, and logs it in memory with a placeholder channel list (`telegram`, `email`); no network call is made. Run on the validation split, this generates 3,611 simulated alerts, matching exactly the combined High+Critical count from Table 20 (3,569 Critical + 42 High).

7.3. Semi-auto IP Block and Exposure Window Simulation

`IPBlockSimulator` triggers only on High/Critical decisions and models the delay of a semi-automatic (analyst-in-the-loop) approval workflow as Delay $\sim$ LogNormal($\mu = 3.0, \sigma = 1.0$) seconds, against a full-automatic baseline with zero delay. This delay-distribution comparison is independent of any specific dataset and was run as a 1,000-sample Monte Carlo simulation for each mode (Table 31), illustrating the intended trade-off: semi-auto review adds a mean exposure window of 36.5 s (p95 = 115.8 s, max = 742.5 s) during which a blocked-pending-review request remains active, versus 0 s for full-auto.

The actual *blocking decision* logic, however, could not be exercised on the validation data: neither BIPIA nor xTRam1 records request-level metadata such as source IP or user agent (Section 5.1), so every row falls back to a placeholder value, which `IPBlockSimulator` explicitly excludes from triggering a block. Consequently, 0 of the 3,611 eligible (High/Critical) rows resulted in a simulated block attempt. We report this plainly as a demonstration-data limitation, not a functional failure of the design: the delay-distribution mechanic above is validated in isolation, but the end-to-end "risk decision → IP block" path has not been exercised against any traffic, real or simulated-with-IPs, in this paper.

Table 31: Simulated exposure window under semi-auto vs. full-auto IP blocking ($n = 1{,}000$ draws per mode; statistical simulation, independent of dataset).

Mode	Mean (s)	p95 (s)	Max (s)
Semi-auto (analyst review)	36.5	115.8	742.5
Full-auto (no delay)	0.0	0.0	0.0

7.4. Attacker Profiling

`AttackerProfiler` is designed to maintain a per-IP threat profile (alert counts, High/Critical trigger frequency, a temporally-decayed weighted score with a 0.95/day decay factor, and a heuristic same-/24-subnet, within-one-hour coordinated-attack flag), aggregated over a 30-day rolling window. For the same reason as Section 7.3 — no genuine source-IP field in BIPIA/xTRam1 — this component profiled 0 distinct IPs on the validation split (all 7,730 rows collapse to a single placeholder address) and consequently could report neither a top-threats ranking nor any coordinated-attack detection. As with IP blocking, this is an architecture presented for future evaluation against traffic that carries genuine request metadata, not a result to be read as validating (or invalidating) the profiling heuristic itself.

7.5. Summary

Table 32 summarizes the readiness of each component. Only decision mapping is validated end-to-end on this paper's data; alerting is demonstrated as working log-generation logic without any live dispatch; IP blocking and attacker profiling remain architecturally specified but, on the two benchmarks used throughout this paper, cannot be exercised beyond their internal, dataset-

independent statistical mechanics (Table 31). This reinforces, with direct evidence rather than by declaration alone, why Sections 7.3–7.4 are scoped as proposed system design (Section 1) and why the IP-reliability caveats discussed in Limitations (Section 9) apply even before considering NAT/VPN/shared-IP effects in a real deployment.

8. Ethics and Privacy Considerations

Both datasets used in this paper (BIPIA [2], xTRam1 [6]) are public, pre-existing research benchmarks containing synthetic or crowd-sourced text; no genuine personal data, private communications, or production traffic was collected, and no human subjects were involved in this study.

The audit-trail schema proposed at Step 1 (Section 4.1) includes `source_ip` and `user_agent` fields, and the proposed Response Layer's IP-blocking and attacker-profiling components (Sections 7.3–7.4) are designed to retain and aggregate this metadata over a 30-day rolling window. Were this system deployed on genuine traffic, such retention would constitute processing of personal data under regimes including the EU GDPR and Vietnam's Decree 13/2023/ND-CP on personal data protection, requiring a documented legal basis, a defined retention limit, and safeguards against re-identification; neither this paper's experiments nor its simulated evaluation (Table 32) implement or test such safeguards, since no real IP/user-agent data was ever processed (Section 7.3). Any production deployment of the Response Layer as designed would need a separate privacy and data-retention review before handling real user metadata, particularly given the known reliability caveats of IP-based blocking and profiling under NAT, VPN, and shared-IP conditions (Section 9), which can cause both misattribution of a block to an innocent user and evasion by an attacker rotating addresses.

The adaptive-attack and cross-lingual evaluations (Section 6.4) were conducted entirely offline, against the authors' own deployed model checkpoints, with no queries issued to any third-party production system; this is a defensive robustness evaluation of the authors' own pipeline, not an attack against external infrastructure.

9. Limitations

9.1. Data and Domain Coverage

All results are derived from two English-language datasets, BIPIA and xTRam1 (Section 5.1); no genuine multilingual, PDF-binary, base64-obfuscated, hidden-CSS, or context-flooding attack samples exist in either source (Table 5). Consequently: (i) the cross-lingual probe (Section 6.4.1) shows recall collapsing to 4.00% on machine-translated Vietnamese attack text, indicating every trainable component of the pipeline is effectively English-only; (ii) the hidden-text similarity threshold and context-flooding ratio threshold (Section 4.2) could only be validated on a real-but-small PDF corpus ($n = 300$) and a synthetic flooding corpus respectively, not on genuine in-the-wild attacks of either kind; (iii) the exact-match blocklist (Section 4.4.1) mined zero entries, so its behavior on more repetitive production traffic (where it is expected to contribute) remains untested; and (iv) baseline comparison (Section 6.1) excludes AgentDojo (an execution framework, not a labeled dataset), PINT (proprietary, no offline access), and PromptArmor (requires GPT-4o API access unavailable to this study).

9.2. Threshold Calibration

Several thresholds are retained at heuristic default values rather than their grid-search optima because the validating corpus was synthetic or too small to trust as a production operating point: the hidden-text similarity threshold (default 0.6 vs. real-corpus optimum 0.95, Table 6) and the context-flooding ratio threshold (default 0.7 vs. synthetic optimum 1.9, Table 8) both likely under-detect at their current settings. The keyword-density/suspicious-score pre-filter mechanism was explicitly *disabled* (Section 4.4.1) after grid search found no operating point simultaneously reaching precision ≥ 0.98 and $n \geq 30$. Eleven of the 20 ATTACK_CUE_PATTERNS entries have validation support < 10 (Section 4.1) and their precision/recall estimates should be treated as provisional. The Risk Scoring `medium_max` boundary (0.62, Section 4.5.6) failed its own monotonicity sanity check (Medium bucket attack rate 87.06% > High bucket 40.48%) and is an acknowledged, unresolved calibration anomaly, not a validated threshold. The OOD Escalation Bonus (+0.15, Section 4.4.4) remains an untuned initial value (Section A). Finally, the Mahalanobis OOD threshold is calibrated purely as a statistical anomaly cutoff (5% false-positive rate on in-distribution validation) in the absence of any labeled OOD ground truth in either dataset, so its correspondence to genuinely out-of-distribution inputs is only indirectly supported (Section 4.4.4) and by the small $n = 13$ escalated-OOD observation in Section 6.1.

9.3. Architectural Components Without Demonstrated Benefit

Two of the pipeline's designed mechanisms did not show a statistically significant contribution on this validation/test data. The Trust-Weighting bridge into the final risk score (Section 4.5) was found, by its own calibrating grid search, to be optimal at scale = 0.00, and the corresponding test-set ablation reproduces this exactly (0 discordant pairs, Table 26); this indicates the *current formula* for injecting source-aware weighting into Risk Scoring is inactive, not that source-aware weighting is unhelpful within Step 3 feature extraction itself, which was not separately ablated. The cas-

Table 32: Response Layer component status, validation split (n = 7,730).

Component	Status	Evidence
Decision mapping	Reproduced, deterministic	100% match, n = 7,730
Alert simulation	Logic demonstrated	3,611 simulated alerts
IP block (delay model)	Mechanic validated	Table 31
IP block (trigger logic)	Not exercised	0/3,611 (no IP field)
Attacker profiling	Not exercised	0 IPs (no IP field)

cade's second escalation stage (BERT+OOD) is not significantly better than trusting DistilBERT alone, on three independent comparisons: the validation-set McNemar test ($p = 0.177$, Section 4.4.7), the test-set ablation ($p = 0.250$, Table 26), and both baseline tables (Section 6.1), where DistilBERT-alone modestly exceeds the full Cascade in F1 on both the in-distribution and OOD benchmarks. A power analysis correctly computed on the true $n = 55$ discordant pairs (Section 4.4.7) shows only 53.4% power to detect a true BERT win-rate of 65%, and 80% power requires a 70% true win-rate — the available sample is too small to rule out even a moderate genuine advantage of the full model. The architecture is retained because it behaves exactly as designed operationally (escalating only the hardest 3.47% of validation traffic, Section 4.4.6) and preserves the system's safety-favoring profile (perfect OOD recall, Table 24), but its accuracy benefit over a single lightweight model is not established on the present data.

9.4. Reproducibility

DistilBERT (notebook 04a) and BERT (notebook 04b) were trained in two independent Kaggle kernel sessions that do not share a working directory; 04b could not locate the cascade threshold persisted by 04a and fell back to a hardcoded default ($\theta = 0.72$), which was ultimately used for all downstream evaluation (Section 4.4.3) rather than the grid-search optimum ($\theta = 0.66$). The deployed value still satisfies the underlying accuracy criterion, so results are valid but more conservative than intended. Separately, the PromptGuard baseline was not evaluated on the OOD benchmark (Section 6.1) due to a variable-scope error in which the malicious-label ID list resolved for the in-distribution evaluation was not carried into the OOD notebook cell. Both gaps are logged here rather than silently patched, and the recommended fix in both cases is to persist configuration/state as a versioned artifact rather than an in-session variable. A related bug was found in the power-analysis cell of notebook 04c itself, which re-derived its sample size from the full escalated-path count ($n = 268$) rather than the actual McNemar discordant-pair count ($n = 55$), overstating the test's power; the corrected computation (Section 4.4.7) is used throughout this paper in place of the notebook's raw printed values.

9.5. Evaluation Scope

Latency (Section 6.5) was measured single-request, batch size 1, with no concurrent load; throughput under concurrent/production load is not measured, and the escalated (`bert_ood`) path is itself nearly 3× slower per request than running BERT alone, so the system-wide latency advantage depends entirely on the escalation rate observed in this data (3.75%) and would erode on a workload with a higher hard-sample rate. The adaptive-attack evaluation (Section 6.4) uses a fixed $n = 50$ subsample and a maximum of 4 adaptive iterations; the observed vulnerability to white-box homoglyph and translation attacks (ASR up to 94%) may under- or over-state real-world exploitability depending on how much query budget and pipeline knowledge a genuine attacker has. The FPR-by-source analysis (Section 6.3) identifies a 7-fold spread across input types but the underlying mechanism (lexical surface features vs. declared source/trust level) is reported as a hypothesis, not a confirmed causal explanation.

9.6. Response Layer

As stated in Section 7, the Response Layer is a proposed design, not an evaluated system. Its IP-blocking and attacker-profiling components could not be exercised at all on either dataset because neither BIPIA nor xTRam1 records genuine request-level metadata (source IP, user agent); every row falls back to a placeholder value, so 0 of 3,611 eligible alerts triggered a block or profiling update (Table 32). Even with real IP data, IP-based blocking and profiling are subject to well-known reliability caveats (NAT, VPN, and shared-IP environments can cause both false attribution and evasion) that have not been modeled here at all.

10. Future Work

Threshold recalibration on larger real corpora. The hidden-text similarity threshold and context-flooding ratio threshold (Section 4.2) are currently retained at conservative defaults below their grid-search optima because the validating corpora were small ($n = 300$) or synthetic; a larger, more diverse real-attack corpus for both would allow these defaults to be raised toward their empirically better-performing values (0.85–0.95 and $\geq$1.5 respectively).

Per-cell grid search of the source-aware weighting table. The weighting table (Table 9, Section 4.3) remains at its originally specified static values; a full grid search over each cell, combined with a redesigned Feature-Signal bridge into Risk Scoring (Section 4.5) that does not collapse to zero contribution, is needed before source-aware weighting's benefit (if any) beyond Step 3 feature extraction can be established.

Resolving the risk-level monotonicity anomaly. The Medium/High risk-tier ordering inversion identified in Section 4.5.6 should be corrected, likely by rerunning Pass 1's F1 optimization jointly across all three thresholds rather than only against the Critical boundary.

Multilingual robustness. Given the 96-percentage-point recall collapse on machine-translated Vietnamese text (Section 6.4.1), extending training data and embeddings to multiple languages is a priority before any non-English deployment.

Adaptive-attack coverage. The current adaptive evaluation uses a fixed $n = 50$ subsample and a 4-round query budget (Section 6.4); a larger sample, a higher iteration budget, and additional attack families (e.g., prompt-in-prompt encoding, multi-turn injection) would give a more complete picture of the pipeline's adaptive attack surface, particularly given the large homoglyph and translation evasion gaps already observed.

A larger hard-subset for cascade evaluation. The corrected power analysis (Section 4.4.7) shows the current $n = 55$ discordant-pair sample cannot rule out a moderate (65%+) true advantage of the BERT+OOD stage; revisiting this comparison on a larger held-out hard subset is needed before concluding the escalation stage should be simplified or removed.

Genuine out-of-distribution ground truth. The Mahalanobis OOD threshold (Section 4.4.4) is calibrated as a statistical anomaly cutoff in the absence of labeled OOD data; evaluating it against genuinely out-of-domain traffic (e.g., other languages, other task domains beyond Open-Prompt-Injection's SST2/SMS-spam pairing) would validate it more directly.

Dedicated FPR error analysis. The mechanism behind the 7-fold FPR spread across input sources (Section 6.3) is reported as a hypothesis; a targeted error analysis of misclassified benign URL and RAG-chunk samples is needed to confirm or refute it.

Broader baseline coverage. AgentDojo, PINT, and PromptArmor (Section 2) were out of scope for this paper (Section 6.1); evaluating against them, where access permits, would strengthen the external validity of the baseline comparison.

Production-realistic latency and metadata. Throughput under concurrent load (Section 6.5) and the Response Layer's IP-dependent components (Section 7) both require evaluation against traffic that carries genuine concurrency and request metadata, neither of which is present in BIPIA/xTRam1.

11. Conclusion

This paper presented a five-stage, dual-stream pipeline for detecting both direct and indirect prompt injection, evaluated end-to-end on two public benchmarks plus an out-of-distribution set and an adaptive attack suite. The full Cascade reaches F1= 0.9834 (accuracy 0.9835) on the locked in-distribution test set and F1= 0.9788 with perfect recall on an out-of-distribution benchmark (Section 6.1), while its three-stage architecture yields a 1.91× system-wide latency improvement over a single full BERT model (Section 6.5).

Consistent with the transparent hyperparameter-justification protocol applied throughout (Appendix A), two design components do not show a demonstrated accuracy benefit on the present data: the Trust-Weighting bridge into Risk Scoring is calibrated to zero contribution by its own grid search (Section 4.5), and the cascade's BERT+OOD escalation stage is not significantly more accurate than DistilBERT alone across three independent comparisons (Sections 4.4.7, 6.2, 6.1), though a corrected power analysis shows the available sample cannot rule out a moderate true advantage (Section 4.4.7). Both components are retained – Trust-Weighting as an active mechanism within Step 3 feature extraction even though its current Risk-Scoring bridge is inactive, and the Cascade for its demonstrated latency benefit and its perfect out-of-distribution recall – rather than removed, but their contribution should be read as architecturally motivated rather than empirically proven on this data. The only ablated component to reach statistical significance is Dual-stream detection ($p = 0.008$, Section 6.2), which is also the mechanism most directly responsible for the pipeline detecting xTRam1's context-free direct injection at all.

Two robustness findings are, in the authors' view, the most operationally important results of this paper, precisely because they are invisible in the static benchmark numbers above: an adaptive, white-box attacker can raise homoglyph-substitution Attack Success Rate from 28% to 84% by iteratively querying the deployed thresholds (Section 6.4), and recall on machine-translated Vietnamese attack text collapses to 4% despite the same text being detected almost perfectly in English (Section 6.4.1). Any deployment of this pipeline, or a similar one, should treat static in-distribution accuracy as a necessary but insufficient indicator of real-world safety, and should budget separately for adaptive-attacker and multilingual evaluation before production use.

Declarations

Competing interests

None of the authors have any competing interests.

Data availability

The BIPIA dataset [2] is publicly available at https://github.com/microsoft/BIPIA. The xTRam1/safe-guard-prompt-injection dataset [6] is publicly available at https://huggingface.co/datasets/xTRam1/safe-guard-prompt-injection. The Open-Prompt-Injection benchmark [1], used for out-of-distribution evaluation (Section 6.1), is publicly available at https://github.com/liu00222/Open-Prompt-Injection. Trained model checkpoints, the combined/processed dataset, and all source code used to produce the results in this paper are available from the authors upon reasonable request.

A. Hyperparameter Justification

Table 33 consolidates every threshold, weight, and constant used in the pipeline, its value, and its validation status, using four tiers: **(V)** validated by grid search against a real, labeled corpus; **(S)** validated only against a synthetic or small proxy corpus; **(H)** a heuristic constant not yet grid-searched; **(D)** explicitly disabled after grid search found no trustworthy operating point.

A.1. Full ATTACK_CUE_PATTERNS Validation

Table 34 reports all 20 patterns in AttackCuePattern, evaluated independently as standalone binary classifiers on the validation split ($n = 7{,}730$), sorted by F1.

Table 33: Consolidated hyperparameter/threshold inventory and validation status.

Parameter	**Value**	**Tier**	**Section**	**Note**
Hidden-text similarity threshold	0.60	S	4.2	Real-corpus optimum 0.95 (F1 0.7075, $n = 300$)
Context-flooding ratio threshold	0.70	S	4.2	Synthetic optimum 1.9 (F1 0.9934, $n = 300$)
Chunking stride / window	50 / 512 tok.	H	4.2	Engineering default, not tuned
`suspicious_pattern_score` weights	0.45/0.35/0.20	H	4.3	Not grid-searched (Eq. 1)
Source-aware weighting table	Table 9	H	4.3	Static; not grid-searched per cell
ATTACK_CUE_PATTERNS (20 phrases)	—	V/H mixed	4.1	5 retained w/ precision ≥ 0.5; 4 removed (P< 0.3); 11 support< 10 (inconclusive)
Exact-match blocklist ($\geq$5 repeats, purity 1.0)	—	H	4.4.1	0 entries mined on this dataset
Regex pre-filter (9 patterns)	—	V	4.4.1	P=1.0000, $n = 23$ on validation
Keyword-threshold pre-filter	$\tau_{kd} = \tau_{susp} = 1.01$	D	4.4.1	Grid search found no cell with P$\geq$ 0.98 & $n \geq 30$
Cascade confidence threshold θ	0.72 (deployed)	V	4.4.3	Grid optimum 0.66; 0.72 used due to session artifact gap
Focal Loss γ	2.0	H	4.4.2	Literature default (Lin et al. [7])
Focal Loss class weight α	data-derived	V	4.4.2	`compute_class_weight("balanced")`, not a free parameter
DistilBERT/BERT lr, batch, epochs	2e-5, 16/8, 3	H	4.4.2	Fixed by compute budget + smoke test
OOD false-positive rate α	5%	H	4.4.4	Statistical convention; no OOD ground truth available
OOD Escalation Bonus	+0.15	H	4.5	**Not yet grid-searched** (open item)
Anomaly Bonus (hidden/deobf./delimiter)	+0.10/ + 0.08/ + 0.08	H	4.5	2 of 3 structurally inactive on this dataset
W_{attack} (Direct/Indirect/Exfil.)	0.6/0.8/1.0	H	4.5	Exfiltration branch never exercised (no such label)
M_{trust} (High/Med/Low/V.Low)	1.0/1.0/1.2/1.4	H	4.5	Not grid-searched
Feature Signal scale	0.00	V	4.5	Grid-searched; monotonically decreasing, optimum at 0
Risk threshold `low_max`	0.49	V	4.5.6	Safety-constrained (missed-attack $\leq$ 1%)
Risk threshold `medium_max`	0.62	H	4.5.6	Midpoint heuristic; **failed monotonicity check**
Risk threshold `high_max`	0.75	V	4.5.6	F1-optimal grid search (F1= 0.9441)
Bootstrap CI (B, α, n cutoff)	1000, 0.05, 100	H	5.2	Standard statistical convention

Table 34: Full ATTACK_CUE_PATTERNS validation (validation set, $n = 7{,}730$), sorted by F1. Four patterns marked † (precision = 0, support = 0) were removed from `classifiers.py`; eleven marked ‡ (support < 10) are flagged as inconclusive.

Pattern	Precision	Recall	F1	Support
"you are"	0.6031	0.0208	0.0401	131
"disregard"	0.9643	0.0071	0.0141	28
"you must"	0.8125	0.0068	0.0135	32
"you will"	0.4407	0.0068	0.0135	59
"as an"	0.6061	0.0053	0.0104	33
"from now on"	1.0000	0.0045	0.0089	17
"forget"	0.9333	0.0037	0.0073	15
"act as"	0.5185	0.0037	0.0073	27
"you are now"	1.0000	0.0034	0.0068	13
"pretend to be"‡	1.0000	0.0016	0.0031	6
"instead of"‡	0.5714	0.0011	0.0021	7
"do not follow"‡	1.0000	0.0008	0.0016	3
"ignore previous"‡	1.0000	0.0003	0.0005	1
"take on the role"‡	1.0000	0.0003	0.0005	1
"role play"‡	1.0000	0.0003	0.0005	1
"new instruction"‡	1.0000	0.0003	0.0005	1
"system prompt"†‡	0.0000	0.0000	0.0000	0
"your new task"†‡	0.0000	0.0000	0.0000	0
"your role is"†‡	0.0000	0.0000	0.0000	0
"assume the role"†‡	0.0000	0.0000	0.0000	0

References

[1] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, 2024.

[2] Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models, 2023.

[3] X. Liu and Y. Ming. Semantic integrity failures in document-to-llm supply chains, 2026.

[4] Meta AI. Prompt guard. Model card, Hugging Face, 2024. https://huggingface.co/meta-llama/Prompt-Guard-86M.

[5] Yupei Liu, Yuqi Jia, Jinyuan Jia, and Neil Zhenqiang Gong. Datasentinel: A game-theoretic detection of prompt injection attacks. arXiv preprint, 2025.

[6] xTRam1. safe-guard-prompt-injection dataset. Hugging Face Datasets, 2024. https://huggingface.co/datasets/xTRam1/safe-guard-prompt-injection.

[7] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

[8] Kimin Lee, Kibok Lee, Honglak Lee, and Jinwoo Shin. A simple unified framework for detecting out-of-distribution samples and adversarial attacks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.